

Jagiellonian University
Department of Theoretical Computer Science

Jakub Wolny

**Implementacja i porównanie algorytmów
wyznaczania wszystkich par ścieżek w grafach**

Praca licencjacka

Promotor: dr hab. inż. Krzysztof Turowski

Wrzesień 2026

Spis treści

1	Wprowadzenie	3
1.1	Sformułowanie problemu i kontekst historyczny	3
1.2	Zastosowania problemu APSP	3
1.3	Algorytm Dijkstry	4
1.3.1	Opis algorytmu	5
1.3.2	Analiza złożoności	6
1.3.3	Zastosowanie do problemu APSP	6
1.4	Algorytm Floyd–Warshalla	6
1.5	Zestawienie algorytmów APSP	7
1.6	Notacja i podstawowe definicje	7
2	Algorytm Spiry	9
2.1	Własności algorytmu Dijkstry	9
2.2	Struktury danych i notacja	9
2.3	Szczegółowy opis algorytmu	10
2.4	Analiza złożoności	11
3	Algorytm Seidela	15
3.1	Poprawność algorytmu	15
3.2	Analiza złożoności	17
4	Algorytm Shoshana–Zwicka	18
4.1	Definicje	18
4.1.1	Iloczyny i macierze indykatorowe	18
4.1.2	Operacje liczbowe na macierzach	19
4.2	Wprowadzenie teoretyczne	19
4.2.1	Redukcja do mnożeń boolowskich	20
4.2.2	Złożoność obliczania iloczynu odległościowego	20
4.2.3	Podjęcie naiwne i jego ograniczenia	21
4.3	Algorytm USP dla grafów nieważonych	21
4.4	Algorytm WUSP dla grafów ważonych	23
4.5	Korekta algorytmu Shoshana–Zwicka	26
5	Algorytm Aingwortha, Chekuriego, Indyka i Motwaniego	27
5.1	Definicje i oznaczenia	27
5.2	Konstrukcja zbioru dominującego	27
5.3	Opis algorytmu Approx-APSP	28
5.4	Dowód poprawności	28
5.5	Analiza złożoności	29

6	Implementacja i porównanie	31
6.1	Wprowadzenie	31
6.2	Szczegóły implementacji	31
6.3	Wyniki	32
6.3.1	Zbiory danych	32
6.3.2	Empiryczna złożoność czasowa	33
6.3.3	Porównanie czasów działania	33
6.3.4	Dokładność aproksymacji Aingwortha	35
6.3.5	Uwagi dotyczące zużycia pamięci	36
6.3.6	Wnioski	36

Rozdział 1

Wprowadzenie

1.1 Sformułowanie problemu i kontekst historyczny

Problem wyznaczania najkrótszych ścieżek między wszystkimi parami wierzchołków (ang. *All-Pairs Shortest Paths*, APSP) należy do fundamentalnych zagadnień teorii grafów i algorytmiki. Zadanie polega na wyznaczeniu dla każdej pary wierzchołków $u, v \in V$ długości $\delta(u, v)$ najkrótszej ścieżki między nimi, rozumianej jako minimum sumy wag krawędzi po wszystkich ścieżkach prowadzących z u do v . W przypadku grafów nieważonych każda krawędź ma wagę 1, a odległość jest po prostu liczbą krawędzi na najkrótszej ścieżce.

Problem APSP jest uogólnieniem problemu znajdowania najkrótszych ścieżek. W problemie pojedynczego źródła (ang. *single-source*) szukamy najkrótszych ścieżek z jednego ustalonego wierzchołka do wszystkich pozostałych – rozwiązuje go m.in. algorytm Dijkstry (dla wag nieujemnych) lub algorytm Bellmana–Forda (dla dowolnych wag). Problem APSP można traktować jako n -krotne uruchomienie algorytmu single-source, co jest naturalnym, choć nie zawsze optymalnym rozwiązaniem.

Pierwsze podejście do problemu APSP w formie bliskiej współczesnej pochodzi z lat 60. XX wieku. W 1959 r. Dijkstra opublikował swój algorytm single-source [1], który stał się podstawą wielu rozwiązań APSP. W 1962 r., Roy oraz Floyd i Warshall niezależnie opisali algorytm programowania dynamicznego rozwiązujący APSP w czasie $\Theta(n^3)$, znany dziś jako algorytm Floyd–Warshalla [2]. Jego prostota oraz możliwość zrównoleglenia obliczeń sprawiają, że nadal pozostaje ważnym punktem odniesienia.

Poniższy przegląd prezentuje cztery algorytmy będące odpowiedzią na potrzebę poprawy złożoności $O(n^3)$ w różnych klasach grafów. Algorytm Spiry [3] obniża oczekiwany czas działania do $O(n^2 \log^2 n)$ dla grafów ważonych z losowymi wagami, stosując posortowane listy krawędzi i struktury turniejowe. Algorytm Seidela [4] rozwiązuje problem dokładnie w czasie $O(n^\omega \log n)$ dla grafów nieskierowanych i nieważonych. Algorytm Shoshana–Zwicka [5] uogólnia ideę Seidela na grafy ważne z wagami ze zbioru $\{1, \dots, M\}$, osiągając czas $O(Mn^\omega)$; omawiamy też korektę błędu znalezionej przez Eirinakisa, Williamsona i Subramaniego [6]. Wreszcie algorytm Aingwortha i in. [7] rezygnuje z dokładności na rzecz szybkości, zwracając aproksymację odległości z błędem addytywnym co najwyżej 2 w czasie $O(n^{2.5} \sqrt{\log n})$, bez korzystania z szybkiego mnożenia macierzy.

1.2 Zastosowania problemu APSP

Problem najkrótszych ścieżek między wszystkimi parami wierzchołków znajduje zastosowanie w wielu dziedzinach, w tym między innymi:

Sieci komputerowe i routing. Protokoły routingu w sieciach komputerowych, takie jak OSPF (ang. *Open Shortest Path First*), opierają się na znajomości najkrótszych ścieżek między węzłami sieci [8]. Każdy router wyznacza tablicę routingu przez obliczenie odległości do wszystkich pozostałych węzłów. Podobne zastosowanie pojawia się w sieciach telekomunikacyjnych, gdzie minimalizacja opóźnienia transmisji sygnału wymaga znajomości najkrótszych tras między dowolnymi parami punktów.

Analiza sieci i miary centralności. W analizie sieci społecznościowych i biologicznych odległości między wierzchołkami służą do wyznaczania miar centralności. Najważniejszą z nich jest centralność pośrednictwa (ang. *betweenness centrality*): dla każdego wierzchołka v mierzy ona, przez jak dużą część wszystkich najkrótszych ścieżek między parami wierzchołków przechodzi v :

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

gdzie σ_{st} oznacza liczbę najkrótszych ścieżek z s do t , a $\sigma_{st}(v)$ – tych spośród nich, które przechodzą przez v [9]. Obliczenie tej miary dla wszystkich wierzchołków wymaga znajomości wszystkich najkrótszych ścieżek w grafie.

Projektowanie układów scalonych. W projektowaniu układów VLSI problem APSP pojawia się przy wyznaczaniu optymalnych połączeń między elementami układu [10]. Minimalizacja długości ścieżek sygnałowych przekłada się bezpośrednio na szybkość działania układu oraz ograniczenie strat energetycznych. Algorytm Dijkstry i jego uogólnienia stanowią rdzeń narzędzi do automatyzacji projektowania układów (ang. *EDA tools*) [10].

Chemia obliczeniowa. Indeks Wienera $W(G)$, definiowany jako suma odległości między wszystkimi parami wierzchołków grafu molekularnego,

$$W(G) = \frac{1}{2} \sum_{u \in V} \sum_{v \in V} \delta(u, v),$$

jest jednym z najważniejszych topologicznych deskryptorów struktury związków chemicznych [8]. Jego obliczenie wymaga rozwiązania problemu APSP dla grafu, w którym wierzchołki odpowiadają atomom, a krawędzie – wiązaniom kowalencyjnym. Indeks ten koreluje z wieloma właściwościami fizykochemicznymi cząsteczek, takimi jak temperatura wrzenia czy ciśnienie pary nasyconej [8].

Przetwarzanie obrazów. Algorytmy segmentacji obrazu oparte na grafach, takie jak metoda „żywego drutu” (ang. *live wire*), wymagają wyznaczania najkrótszych ścieżek w grafach pikseli [8]. W robotyce problem APSP stosowany jest do planowania tras w przestrzeni konfiguracyjnej, gdzie węzły odpowiadają możliwym pozycjom robota, a krawędzie – dozwolonym przejściom [8].

1.3 Algorytm Dijkstry

Algorytm Dijkstry [1] rozwiązuje problem najkrótszych ścieżek z jednego źródła (ang. *single-source shortest paths*, SSSP) w grafach z nieujemnymi wagami krawędzi. Stanowi

on podstawę algorytmu Spiry [3] oraz naturalny punkt wyjścia w rozwiązywaniu problemu APSP.

Algorytm Dijkstry wyznacza najkrótsze ścieżki z ustalonego wierzchołka źródłowego s do wszystkich pozostałych wierzchołków grafu. Opiera się on na podejściu zachłannym: w każdym kroku algorytm wybiera wierzchołek u spoza zbioru już przetworzonych, dla którego bieżące oszacowanie odległości $d[u]$ jest najmniejsze, a następnie *relaksuje* wszystkie krawędzie wychodzące z u .

Kluczowa obserwacja, na której opiera się poprawność algorytmu, jest następująca: jeżeli wagi krawędzi są nieujemne, to w chwili wybrania wierzchołka u jego oszacowanie $d[u]$ jest już ostateczne i równa się faktycznej minimalnej odległości od s (będziemy ją oznaczać przez δ). Wynika to stąd, że każda inna ścieżka z s do u , przechodząca przez wierzchołek dotychczas nieprzetworzony, musi mieć długość co najmniej $d[u]$ – ponieważ każda kolejna krawędź na tej ścieżce ma wagę nieujemną, a więc nie może skrócić drogi.

Lemat 1.3.1. *Niech S oznacza zbiór wierzchołków ostatecznie przetworzonych przez algorytm Dijkstry. W chwili dodania wierzchołka u do S zachodzi $d[u] = \delta(s, u)$.*

Dowód. Dowodzimy przez indukcję po $|S|$. Dla $|S| = 1$ mamy $S = \{s\}$ i $d[s] = 0 = \delta(s, s)$.

Niech u będzie wierzchołkiem dodawanym do S w kroku $|S| = k + 1$. Rozważmy dowolną ścieżkę p z s do u w grafie. Niech y będzie pierwszym wierzchołkiem na p spoza S , a x – jego poprzednikiem na p (wierzchołek $x \in S$). Z hipotezy indukcyjnej $d[x] = \delta(s, x)$, a po relaksacji krawędzi (x, y) mamy $d[y] \leq \delta(s, x) + w(x, y) = \delta(s, y)$.

Ponieważ wagi krawędzi są nieujemne, długość podścieżki z y do u na p jest nieujemna, więc $\delta(s, y) \leq \delta(s, u)$. Z kolei $d[u] \leq d[y] \leq \delta(s, u)$, bo u zostało wybrane jako minimum. Jednocześnie $d[u] \geq \delta(s, u)$, ponieważ $d[u]$ jest długością pewnej ścieżki z s do u . Stąd $d[u] = \delta(s, u)$. $\square$

1.3.1 Opis algorytmu

Algorytm przyjmuje graf $G = (V, E)$ z funkcją wag $w: E \rightarrow \mathbb{R}_{\geq 0}$ oraz wierzchołek źródłowy s .

Algorytm 1 Algorytm Dijkstry – najkrótsze ścieżki z jednego źródła

Wejście: Graf $G = (V, E)$, wagi nieujemne, źródło $s \in V$

Wyjście: Tablica d : $d[v] = \delta(s, v)$ dla każdego $v \in V$

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow +\infty$  dla  $v \neq s$ 
2:  $Q \leftarrow V$  ▷ kolejka priorytetowa
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \arg \min_{v \in Q} d[v]$ 
5:    $Q \leftarrow Q \setminus \{u\}$ 
6:   for all  $v$  sąsiada  $u$  do
7:     if  $d[u] + w(u, v) < d[v]$  then
8:        $d[v] \leftarrow d[u] + w(u, v)$ 
9:     end if
10:  end for
11: end while
12: return  $d$ 
```

1.3.2 Analiza złożoności

Złożoność algorytmu Dijkstry zależy od implementacji kolejki priorytetowej Q .

Kopiec binarny Wyciągnięcie minimum kosztuje $O(\log n)$, co wykonujemy n razy. Każda z m relaksacji może wymagać operacji DECREASE-KEY w czasie $O(\log n)$. Całkowita złożoność: $O((n + m) \log n)$.

Kopiec Fibonacciego Operacja DECREASE-KEY działa w zamortyzowanym czasie $O(1)$, a wyciągnięcie minimum w $O(\log n)$. Całkowita złożoność: $O(m + n \log n)$ [11]. Jest to optymalne dla grafów rzadkich.

1.3.3 Zastosowanie do problemu APSP

Uruchamiając algorytm Dijkstry niezależnie dla każdego z n wierzchołków jako źródła, otrzymujemy rozwiązanie problemu APSP. Dla grafów gęstych ($m = \Theta(n^2)$) każde z powyższych podejść daje złożoność $O(n^3)$, co jest zbieżne ze złożonością algorytmu Floyda–Warshalla. Przewaga podejścia opartego na Dijkstrze ujawnia się dla grafów rzadkich ($m = o(n^2)$): przy kopcu Fibonacciego uzyskujemy $O(nm + n^2 \log n)$, co dla $m = O(n)$ redukuje się do $O(n^2 \log n)$, czyli wyniku lepszego niż $O(n^3)$.

Istotnym ograniczeniem jest wymóg nieujemności wag krawędzi. W przypadku grafów z ujemnymi wagami (ale bez ujemnych cykli) stosuje się algorytm Bellmana–Fordy [12] jako procedurę single-source, lub wstępne przepróbkowanie wag metodą Johnsona [13], która sprowadza problem do n uruchomień Dijkstry na grafie z przeliczonymi (nieujemnymi) wagami.

Algorytm Dijkstry jest również punktem wyjścia dla algorytmu Spiry [3], który przez wstępne posortowanie list krawędzi i inteligentniejszy wybór kandydatów do relaksacji osiąga oczekiwany czas $O(n^2 \log^2 n)$ dla grafów z losowymi wagami.

1.4 Algorytm Floyda–Warshalla

Algorytm Floyda–Warshalla [2] to metoda programowania dynamicznego rozwiązująca problem APSP w grafach z dowolnymi wagami krawędzi, ale bez cykli ujemnych. Jego złożoność czasowa wynosi $\Theta(n^3)$.

Idea algorytmu opiera się na następującej obserwacji: najkrótsza ścieżka z i do j przechodząca wyłącznie przez wierzchołki ze zbioru $\{1, \dots, k\}$ albo nie przechodzi przez wierzchołek k (i jest wówczas taka sama jak najkrótsza ścieżka przez zbiór $\{1, \dots, k-1\}$), albo przechodzi przez k i rozkłada się na dwie podścieżki: z i do k oraz z k do j , obie w zbiorze $\{1, \dots, k-1\}$. Formalnie:

$$d^{(k)}(i, j) = \min\{d^{(k-1)}(i, j), d^{(k-1)}(i, k) + d^{(k-1)}(k, j)\},$$

z warunkiem brzegowym $d^{(0)}(i, j) = w(i, j)$ (waga krawędzi lub $+\infty$, gdy krawędź nie istnieje). Po wykonaniu n iteracji po k macierz $D^{(n)}$ zawiera wszystkie odległości. Istotną zaletą jest, że algorytm wymaga jedynie stałej dodatkowej liczby pamięci.

Algorytm Floyda–Warshalla jest szczególnie odpowiedni dla grafów gęstych ($m = \Theta(n^2)$), dla których jego złożoność $O(n^3)$ jest porównywalna do podejścia polegającego na n -krotnym uruchomieniu algorytmu Dijkstry z kopcem Fibonacciego, o złożoności $O(n^2 \log n + nm)$. Jego praktyczną zaletą jest również regularny sposób dostępu do danych, sprzyjający dobrej lokalności pamięci.

1.5 Zestawienie algorytmów APSP

Poniższa tabela zbiera kilka istotnych algorytmów dla problemu APSP, ze szczególnym uwzględnieniem czterech omawianych w pracy. Kolumna *Skierowany* oznacza, czy algorytm działa dla grafów skierowanych, kolumna *Wagi* precyzuje dopuszczalny zakres wag krawędzi, a kolumna *Błąd* wskazuje jaki maksymalny błąd może osiągnąć algorytm.

Algorytm	Skierowany	Wagi	Złożoność	Błąd
Floyd–Warshall (1962)	tak	dowolne	$O(n^3)$	0
Spira (1973)	nie	dodatnie	$O(n^2 \log^2 n)$ śr.	0
Johnson (1977)	tak	dowolne	$O(n^2 \log n + nm)$	0
Tarjan [†]	tak	dodatnie	$O(n^2 \log n + nm)$	0
Seidel (1995)	nie	nieważony	$O(n^\omega \log n)$	0
Shoshan–Zwick (1999)	nie	$\{1, \dots, M\}$	$O(Mn^\omega)$	0
Williams (2014)	tak	dowolne	$O\left(n^3 / 2^{\Omega(\sqrt{\log n})}\right)$	0
Aingworth i in. (1999)	nie	nieważony	$O(n^{2.5} \sqrt{\log n})$	+2
Dor–Halperin–Zwick (2000)	nie	nieważony	$\tilde{O}(n^{7/3})$	+2

[†]Tarjan [11]: algorytm Dijkstry z kopcem Fibonacciego.

Tabela 1.1: Porównanie wybranych algorytmów dla problemu APSP. Pogrubione wiersze odpowiadają algorytmom szczegółowo omawianym w pracy. Przez m oznaczono liczbę krawędzi, a $\tilde{O}$ pomija czynniki polilogarytmiczne. Przez ω oznaczamy najmniejszą liczbę rzeczywistą taką, że istnieje algorytm mnożenia dwóch macierzy rozmiaru $n \times n$ działający w czasie $O(n^{\omega+\varepsilon})$ dla dowolnego $\varepsilon > 0$. Aktualnie najlepsze znane ograniczenie wynosi $\omega < 2.371339$ [14].

Warto zauważyć kilka kluczowych tendencji widocznych w tabeli. Algorytmy ogólne (Floyd–Warshall, Johnson) działają dla szerokiej klasy grafów, lecz ich złożoność jest ograniczona przez barierę $O(n^3)$ dla gęstych grafów. Specjalizacja do grafów nieskierowanych pozwala na wykorzystanie symetrii odległości ($\delta(u, v) = \delta(v, u)$), co jest kluczowe dla algorytmów Seidela i Shoshana–Zwicka – nie jest znana analogiczna technika dla grafów skierowanych. Wreszcie rezygnacja z dokładności (algorytm Aingwortha i in.) pozwala całkowicie uniknąć kosztownego mnożenia macierzy i osiągnąć złożoność podkwadratową w n^2 czynnikach.

1.6 Notacja i podstawowe definicje

W tym podrozdziale przedstawimy podstawowe definicje i oznaczenia używane w dalszej części pracy. Przyjmujemy następującą konwencję: $\delta(u, v)$ oznacza zawsze funkcję odległości zwracającą najkrótszą drogę w grafie, natomiast zapisy postaci d_{ij} czy D_{ij} rezerwujemy dla zapisów konkretnych macierzy pojawiających się w algorytmach.

Definicja 1.6.1 ([15], Graf ważony nieskierowany). Grafem ważonym nieskierowanym nazywamy trójkę $G = (V, E, w)$, gdzie:

- V to skończony, niepusty zbiór wierzchołków,
- $E \subseteq \{\{u, v\} : u, v \in V, u \neq v\}$ – zbiór krawędzi nieskierowanych,

- $w: E \rightarrow \mathbb{R}$ – funkcja wag przypisująca każdej krawędzi wartość rzeczywistą.

Dla wygody zapisujemy $w(u, v)$ zamiast $w(\{u, v\})$. Liczbę wierzchołków oznaczamy przez $n = |V|$, a liczbę krawędzi przez $m = |E|$.

Definicja 1.6.2 ([15], Graf ważony skierowany). Grafem ważonym skierowanym nazywamy trójkę $G = (V, E, w)$, gdzie:

- V to skończony, niepusty zbiór wierzchołków,
- $E \subseteq V \times V$ – zbiór krawędzi skierowanych (par uporządkowanych),
- $w: E \rightarrow \mathbb{R}$ – funkcja wag przypisująca każdej krawędzi skierowanej wartość rzeczywistą.

Dla wygody zapisujemy $w(u, v)$ zamiast $w((u, v))$.

Graf nieskierowany można traktować jako szczególny przypadek grafu skierowanego, zastępując każdą krawędź nieskierowaną $\{u, v\}$ parą krawędzi skierowanych (u, v) i (v, u) o tej samej wadze [15]. Graf nazywamy *nieważonym*, gdy $w(e) = 1$ dla każdej krawędzi $e \in E$.

Definicja 1.6.3 ([16], Funkcja odległości). Dla pary wierzchołków $u, v \in V$ przez $\delta(u, v)$ oznaczamy długość najkrótszej ścieżki z u do v , czyli

$$\delta(u, v) = \min \left\{ \sum_{e \in P} w(e) : P \text{ jest ścieżką z } u \text{ do } v \right\}.$$

W przypadku grafu nieważonego $\delta(u, v)$ jest liczbą krawędzi na najkrótszej ścieżce. Przyjmujemy $\delta(v, v) = 0$ oraz $\delta(u, v) = +\infty$, gdy v jest nieosiągalne z u .

Definicja 1.6.4 ([15], Stopień wierzchołka). Stopniem wierzchołka v w grafie nieskierowanym, oznaczanym przez d_v , nazywamy liczbę krawędzi incydentnych z v .

Definicja 1.6.5 ([7], Sąsiedztwo). Dwa wierzchołki u, v grafu G nazywamy sąsiednimi, jeśli $\{u, v\} \in E$. Zbiór wszystkich sąsiadów wierzchołka v oznaczamy przez $N(v)$. W pracy przyjmujemy $N(v) = \{u \in V : \{u, v\} \in E\} \cup \{v\}$.

Definicja 1.6.6 ([7], Uogólnione sąsiedztwo). Dla $k \geq 1$ definiujemy k -sąsiedztwo wierzchołka v jako

$$N_k(v) = \{u \in V : 0 < \delta(v, u) \leq k\}.$$

Uwaga: Zauważmy, że zgodnie z definicją $N(v) \neq N_1(v)$.

Rozdział 2

Algorytm Spiry

2.1 Własności algorytmu Dijkstry

P. M. Spira w swojej pracy [3] zaproponował algorytm obniżający tę złożoność do średniego czasu $O(n^2 \log^2 n)$. Punktem wyjścia dla algorytmu Spiry jest kluczowa obserwacja dotycząca zachowania klasycznego algorytmu Dijkstry (por. 1.3), wyrażona w poniższym lemacie:

Lemat 2.1.1 ([3, Lemat 3.1]). *Założmy, że najkrótsze ścieżki z wierzchołka źródłowego do pewnego podzbioru wierzchołków $i_1, i_2, \dots, i_k$ zostały już ostatecznie wyznaczone przez algorytm Dijkstry. Wówczas następnym wierzchołkiem, do którego zostanie znaleziona najkrótsza ścieżka, będzie ten nieodwiedzony wierzchołek, który znajduje się najbliżej wierzchołka źródłowego lub najbliżej jednego z już odwiedzonych wierzchołków $i_1, \dots, i_k$.*

Powyższy lemat sugeruje, że nie ma potrzeby analizowania wszystkich krawędzi wychodzących z odwiedzonych wierzchołków jednocześnie. Zamiast tego, korzystne okazuje się wcześniejsze posortowanie krawędzi dla każdego wierzchołka i sprawdzanie ich zaczynając od najkrótszej.

2.2 Struktury danych i notacja

Definicja 2.2.1 ([3], Rozegrane drzewo binarne). *Rozegranym drzewem binarnym nazywamy drzewo binarne, w którym niektóre liście zawierają liczby rzeczywiste, a inne są puste. Każdy węzeł wewnętrzny posiadający przynajmniej jednego niepustego potomka przyjmuje wartość równą najmniejszej z wartości swoich potomków. W konsekwencji wartość w korzeniu drzewa jest minimum spośród wszystkich wartości w liściach.*

Definicja 2.2.2 ([3], Model losowy). *Mówimy, że graf spełnia założenie modelu losowego, jeżeli jego wagi krawędzi są niezależnymi zmiennymi losowymi pochodzącymi z identycznego, nieujemnego rozkładu prawdopodobieństwa.*

Aby formalnie uzasadnić efektywność wyboru kolejnych wierzchołków, Spira odwołuje się do specyficznej struktury danych – tzw. rozegranego drzewa binarnego (ang. *played binary tree*), które modeluje działanie drzewa turniejowego.

Zbudowanie rozegranego drzewa binarnego dla n elementów umieszczonych w liściach wymaga $n - 1$ porównań; po jego skonstruowaniu odczytanie pierwszego minimum jest natychmiastowe ($O(1)$), gdyż wartość ta znajduje się w korzeniu. Wyznaczenie każdego kolejnego minimum – poprzez usunięcie dotychczasowego zwycięzcy i zaktualizowanie ścieżki

od jego liścia do korzenia – wymaga jedynie $\lceil \log_2 n \rceil$ dodatkowych porównań. Z punktu widzenia działania algorytmu najkrótszych ścieżek, Spira opiera się na następującym twierdzeniu:

Twierdzenie 2.2.3. *Niech $\{S_i\}_{i=1,\dots,k}$ będzie rodziną zbiorów liczb rzeczywistych, taką że:*

- (a) $S_{i-1} \setminus S_i = \min S_{i-1}$,
- (b) $1 \leq |S_i \setminus S_{i-1}| \leq 2$,
- (c) $|S_i| \leq n$.

Wówczas istnieje algorytm, który znajduje kolejne minima zbiorów $S_1, S_2, \dots, S_k$ przy użyciu co najwyżej $2k \lceil \log_2 n \rceil$ porównań.

Ze względów praktycznych, zamiast implementować bezpośrednio strukturę rozegranego drzewa binarnego, w niniejszej pracy stosujemy kopiec binarny typu minimum (ang. *binary min-heap*) [16]. Zapewnia on identyczne asymptotyczne gwarancje czasowe wymagane przez powyższe twierdzenie.

2.3 Szczegółowy opis algorytmu

Algorytm rozwiązuje problem APSP niezależnie dla każdego wierzchołka źródłowego. Zakładamy, że krawędzie wychodzące z każdego wierzchołka zostały wstępnie posortowane w czasie $O(n^2 \log n)$.

Rozważmy pojedyncze źródło (oznaczymy je jako węzeł 1). Algorytm buduje zbiór już zidentyfikowanych najkrótszych ścieżek do wierzchołków (bez straty ogólności) $1, 2, \dots, k$, których odległości wynoszą odpowiednio $D_{11} = 0, D_{12}, \dots, D_{1k}$. Niech d_{i,m_i} oznacza najkrótszą jeszcze niewykorzystaną krawędź wychodzącą z wierzchołka i , prowadzącą do wierzchołka m_i . Algorytm utrzymuje w pamięci zbiór kandydujących ścieżek i w każdym kroku wyznacza:

$$\min\{D_{1i} + d_{i,m_i} : 1 \leq i \leq k\}.$$

Założmy, że powyższe minimum przyjmuje wartość $D_{1j} + d_{jm_j}$ czyli zwycięża krawędź z wierzchołka j do m_j . Występują wówczas dwa przypadki:

1. **Węzeł m_j był już wcześniej odwiedzony:** Ścieżka nie wnosi nowej wartości. Krawędź d_{jm_j} zostaje trwale odrzucona. W jej miejsce algorytm pobiera z posortowanej listy węzła j kolejną krawędź (oznaczymy ją $d_{jm'_j}$) i aktualizuje zbiór kandydatów wartością $D_{1j} + d_{jm'_j}$.
2. **Węzeł m_j nie był odwiedzony:** Algorytm oznacza wierzchołek m_j , przypisując mu odległość $D_{1m_j} = D_{1j} + d_{jm_j}$. Ponieważ wykorzystano krawędź wychodzącą z j , algorytm aktualizuje zbiór dostępnych krawędzi, wstawiając nową wartość $D_{1j} + d_{jm'_j}$. Dodatkowo, nowo odkryty wierzchołek m_j zyskuje prawo udziału w poszukiwaniach: algorytm pobiera pierwszą (najkrótszą) krawędź wychodzącą z m_j (powiedzmy d_{m_js}) i wstawia do zbioru kandydatów wartość $D_{1m_j} + d_{m_js}$.

Dzięki strukturze opisanej w powyższym twierdzeniu, każda ewaluacja nowego minimum zajmuje czas $O(\log n)$.

Algorytm 2 Algorytm Spiryl – APSP w grafie ważonym (dla jednego źródła)

Wejście: Graf ważony $G = (V, E)$

Wyjście: Tablica odległości D

```
1: Posortuj rosnąco zbiory krawędzi  $\{d_{ij} : 1 \leq j \neq i \leq n\}$  dla każdego  $i$ 
2: Ustaw  $I(i, j) \leftarrow \ell$ , gdzie  $d_{i\ell}$  jest  $j$ -tym elementem posortowanej listy krawędzi z  $i$ 
3: Ustaw  $D_{ii} \leftarrow 0$  dla  $1 \leq i \leq n$ 
4: Ustaw  $\text{LABEL}(i, j) \leftarrow \text{NIE}$  dla  $i \neq j$ ;  $\text{LABEL}(i, i) \leftarrow \text{TAK}$ 
5: for  $i = 1$  to  $n$  do
6:   Ustaw  $p_j \leftarrow 1$  dla  $1 \leq j \leq n$ 
7:    $\text{COUNT} \leftarrow 1$ ;  $k \leftarrow i$ 
8:    $s_k \leftarrow D_{ik} + d_{k, I(k, 1)}$ 
9:    $p_k \leftarrow p_k + 1$ ;  $S \leftarrow S \cup \{s_k\}$ 
10:  repeat
11:     $t \leftarrow \arg \min\{s_j : s_j \in S\}$ ;  $p_t \leftarrow p_t + 1$ 
12:    if  $p_t = n + 1$  then break
13:    end if
14:     $s_t \leftarrow D_{it} + d_{t, I(i, p_t)}$ 
15:    if  $\text{LABEL}(i, I(i, p_t - 1)) = \text{TAK}$  then continue
16:    end if
17:     $\text{LABEL}(i, I(i, p_t - 1)) \leftarrow \text{TAK}$ 
18:     $D_{i, I(i, p_t - 1)} \leftarrow D_{it} + d_{t, I(i, p_t - 1)}$ 
19:     $k \leftarrow I(i, p_t - 1)$ ;  $\text{COUNT} \leftarrow \text{COUNT} + 1$ 
20:     $s_k \leftarrow D_{ik} + d_{k, I(k, 1)}$ ;  $S \leftarrow S \cup \{s_k\}$ 
21:  until  $\text{COUNT} > n$ 
22: end for
23: return  $D$ 
```

2.4 Analiza złożoności

Analiza probabilistyczna wykazuje, że zaproponowana metoda przeszukiwania wykonuje średnio znacznie mniej pracy niż pełne przeszukanie Dijkstry. Analizę opieramy na założeniu losowego modelu grafu.

Założenie to pociąga za sobą kluczową konsekwencję: posortowanie list krawędzi wychodzących z poszczególnych wierzchołków nie dostarcza informacji o relatywnej kolejności odległości do pozostałych – jeszcze nieodwiedzonych – węzłów. Każdy z pozostałych wierzchołków z równym prawdopodobieństwem może okazać się celem krawędzi wylosowanej jako kolejna z posortowanej listy.

Łatwo zauważyć, że wyznaczanie minimum w zbiorze kandydatów jest krokiem dominującym, jeśli chodzi o złożoność. Niech N_{ij} oznacza liczbę operacji wyciągnięcia elementu z kopca, z wierzchołka źródłowego i , w momencie gdy $j - 1$ najkrótszych ścieżek zostało już znalezionych.

Lemat 2.4.1. *W modelu losowym oczekiwana liczba operacji wyciągnięcia elementu z kopca spełnia:*

$$\mathbb{E}[N_{ij}] \leq \frac{n}{n - j}.$$

Dowód. Prawdopodobieństwo, że kolejna badana krawędź dociera do węzła dotychczas nieodwiedzanego, jest ograniczone z dołu przez stosunek liczby nieodwiedzonych wierz-

chołków do wszystkich możliwych celów:

$$P(\text{odwiedzenie nowego wężła}) \geq \frac{n-j}{n}.$$

Zgodnie z własnościami rozkładu geometrycznego, wartość oczekiwana zmiennej opisującej liczbę prób do momentu pierwszego sukcesu ograniczona jest przez:

$$\mathbb{E}[N_{ij}] \leq \frac{n-j}{n} \sum_{m=1}^{\infty} m \left(\frac{j}{n}\right)^{m-1} = \frac{n-j}{n} \cdot \frac{1}{\left(1 - \frac{j}{n}\right)^2} = \frac{n-j}{n} \cdot \frac{n^2}{(n-j)^2} = \frac{n}{n-j}.$$

□

Lemat 2.4.2. *Oczekiwana liczba operacji na kopcu potrzebna do wyznaczenia wszystkich najkrótszych ścieżek z pojedynczego źródła wynosi $O(n \log n)$.*

Dowód. Aby algorytm zakończył działanie dla jednego źródła, należy zsumować pracę potrzebną do znalezienia każdego z $n-1$ pozostałych wierzchołków w grafie:

$$\sum_{j=1}^{n-1} \mathbb{E}[N_{ij}] \leq \sum_{j=1}^{n-1} \frac{n}{n-j} = n \sum_{k=1}^{n-1} \frac{1}{k} = O(n \log n).$$

□

Twierdzenie 2.4.3. *W modelu losowym oczekiwana złożoność algorytmu Spiry dla problemu APSP wynosi $O(n^2 \log^2 n)$.*

Dowód. Z poprzedniego lematu, oczekiwana liczba operacji na kopcu dla jednego źródła wynosi $O(n \log n)$. Każda modyfikacja kopca pochłania $O(\log n)$ operacji, a cała procedura wykonywana jest niezależnie dla wszystkich n źródeł, zatem:

$$M_n \leq C \cdot n \cdot \left(n \sum_{k=1}^{n-1} \frac{1}{k} \right) \cdot \log n = O(n^2 \log^2 n),$$

gdzie C jest stałą wynikającą z kosztu operacji na wybranej strukturze danych.

□

Analiza wariancji

Spira udowodnił również ograniczenie na wariancję procesu, wykazując, że odchylenie standardowe liczby kroków algorytmu wynosi co najwyżej $O(n^2 \log n)$.

Z analizy wartości oczekiwanej wiemy, że w modelu losowym prawdopodobieństwo tego, że pojedyncza operacja prowadzi do wierzchołka dotychczas nieodwiedzonego, wynosi co najmniej $p = (n-j)/n$, a zmienna N_{ij} ma rozkład geometryczny z prawdopodobieństwem sukcesu p .

Dla zmiennej losowej X o rozkładzie geometrycznym z parametrem p drugi moment wynosi:

$$\mathbb{E}[X^2] = \frac{2-p}{p^2}.$$

Stąd, korzystając z nierówności

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \leq \mathbb{E}[X^2],$$

otrzymujemy:

$$\text{Var}(N_{ij}) \leq \mathbb{E}[N_{ij}^2] \leq \frac{2-p}{p^2} = \frac{2 - \frac{n-j}{n}}{\left(\frac{n-j}{n}\right)^2} = \frac{n(n+j)}{(n-j)^2} \leq \frac{2n^2}{(n-j)^2},$$

gdzie ostatnia nierówność wynika z $n+j \leq 2n$.

Rozważmy wariancję dla ustalonego źródła. Niech $N_i = \sum_{j=1}^{n-1} N_{ij}$ będzie łączną liczbą kroków dominujących dla źródła i . Aby obliczyć $\text{Var}(N_i)$, musimy rozstrzygnąć kwestię korelacji między zmiennymi N_{ij} dla różnych j .

Rozważmy zmienną N_{ij} , która opisuje liczbę prób potrzebnych do znalezienia j -tego nowego wierzchołka, po tym, jak $(j-1)$ -szy został już znaleziony. Warunkowo, przy ustalonym zbiorze $j-1$ już odwiedzonych wierzchołków, zmienna N_{ij} zależy wyłącznie od rozkładu celów krawędzi jeszcze nieprzebadanych. W modelu losowym te cele są jednakowo prawdopodobnymi permutacjami wierzchołków (por. [3], sekcja 4), więc rozkład N_{ij} warunkowy względem wyniku faz $1, \dots, j-1$ (wcześniejszych) zależy jedynie od liczby nieodwiedzonych wierzchołków, która jest deterministyczna i równa $n-j$. Oznacza to, że N_{ij} jest warunkowo niezależna od $N_{i1}, \dots, N_{i,j-1}$ przy ustalonym zbiorze odwiedzonych wierzchołków.

Korzystając ze wzoru na sumę wariancji mamy:

$$\text{Var}(N_i) = \text{Var}\left(\sum_{j=1}^{n-1} N_{ij}\right) = \sum_{j=1}^{n-1} \text{Var}(N_{ij}) + 2 \sum_{j < k} \text{Cov}(N_{ij}, N_{ik}).$$

Ponieważ fazy algorytmu są sekwencyjne, a każda faza zaczyna się od stanu w pełni zeterminowanego przez wynik fazy poprzedniej (zbiór odwiedzonych wierzchołków i pozycje wskaźników w posortowanych listach), składniki kowariancji znikają, czyli $\text{cov}(N_{ij}, N_{ik}) = 0$ dla $j \neq k$. Zatem:

$$\text{Var}(N_i) = \sum_{j=1}^{n-1} \text{Var}(N_{ij}) \leq \sum_{j=1}^{n-1} \frac{2n^2}{(n-j)^2} = 2n^2 \sum_{k=1}^{n-1} \frac{1}{k^2}.$$

Ponieważ $\sum_{k=1}^{\infty} 1/k^2 = \pi^2/6 < 2$, otrzymujemy:

$$\text{Var}(N_i) < 2n^2 \cdot 2 = O(n^2).$$

Niech $N = \sum_{i=1}^n N_i$ będzie całkowitą liczbą kroków dominujących po wszystkich n źródłach. Zmienne N_i i N_j dla $i \neq j$ nie są niezależne: ten sam losowy graf jest używany dla wszystkich źródeł jednocześnie. Nie jest to jednak przeszkodą, ponieważ możemy ograniczyć wariancję sumy bez zakładania niezależności, korzystając z nierówności Cauchy'ego-Schwarza: dla dowolnych zmiennych losowych X, Y zachodzi

$$\mathbb{E}[XY] \leq \sqrt{\mathbb{E}[X^2] \mathbb{E}[Y^2]}.$$

Z symetrii modelu losowego (wszystkie źródła mają identyczny rozkład) zachodzi $\mathbb{E}[N_i^2] = \mathbb{E}[N_1^2]$ dla każdego i . Szacujemy wariancję N przez jego drugi moment:

$$\begin{aligned} \mathbb{E}[N^2] &= \mathbb{E}\left[\left(\sum_{i=1}^n N_i\right)^2\right] = \sum_{i=1}^n \mathbb{E}[N_i^2] + \sum_{\substack{i,j=1 \\ i \neq j}}^n \mathbb{E}[N_i N_j] \\ &= n \mathbb{E}[N_1^2] + n(n-1) \mathbb{E}[N_1 N_2]. \end{aligned}$$

Z nierówności Cauchy’ego–Schwarza:

$$\mathbb{E}[N_1 N_2] \leq \sqrt{\mathbb{E}[N_1^2] \mathbb{E}[N_2^2]} = \mathbb{E}[N_1^2],$$

a zatem:

$$\mathbb{E}[N^2] \leq n \mathbb{E}[N_1^2] + n(n-1) \mathbb{E}[N_1^2] = n^2 \mathbb{E}[N_1^2].$$

Drugi moment N_1 wyrażamy przez wariancję i kwadrat średniej:

$$\mathbb{E}[N_1^2] = \text{Var}(N_1) + (\mathbb{E}[N_1])^2 \leq O(n^2) + O(n^2 \log^2 n) = O(n^2 \log^2 n).$$

Składając te oszacowania:

$$\text{Var}(N) \leq \mathbb{E}[N^2] \leq n^2 \cdot O(n^2 \log^2 n) = O(n^4 \log^2 n).$$

Odchylenie standardowe liczby kroków dominujących wynosi zatem co najwyżej $O(n^2 \log n)$. Ponieważ każdy krok dominujący wymaga $O(\log n)$ operacji na kopcu, łączna liczba operacji $M_n = N \cdot O(\log n)$ ma odchylenie standardowe:

$$\sigma(M_n) = O(\log n) \cdot \sigma(N) = O(n^2 \log^2 n).$$

Ponieważ wariancja M_n jest skończona, na mocy nierówności Czebyszewa prawdopodobieństwo, że algorytm wykona więcej niż $\mathbb{E}[M_n] + k \sigma(M_n)$ operacji, wynosi co najwyżej $1/k^2$. Pozwala to połączyć algorytm Spiry ze standardowym algorytmem $O(n^3)$: uruchamiamy oba równolegle i przerywamy algorytm Spiry, jeśli nie zakończy się w czasie $\mathbb{E}[M_n] + k \sigma(M_n)$. Wybierając $k = \sqrt{n}$, prawdopodobieństwo przerwania wynosi co najwyżej $1/n$, oczekiwany czas działania pozostaje $O(n^2 \log^2 n)$, a pesymistyczny czas całej procedury spada do $O(n^3)$.

Złożoność pesymistyczna. Uzyskany wynik $O(n^2 \log^2 n)$ ma charakter średni i nie stanowi gwarancji dla dowolnego wejścia. Jeżeli założenie o losowej kolejności napotykanym celów krawędzi przestaje obowiązywać, kolejne krawędzie zdejmowane z kopca mogą wielokrotnie prowadzić do wierzchołków już odwiedzonych, nie powodując odkrycia nowego dystansu.

Dla ustalonego źródła każda lista krawędzi wychodzących może w najgorszym przypadku zostać przeskanowana niemal w całości, co daje $O(m)$ rozważonych krawędzi zamiast oczekiwanych $O(n \log n)$. Ponieważ każda operacja na drzewie rozegranym wymaga $O(\log n)$ czasu, pesymistyczny koszt dla jednego źródła wynosi $O(m \log n)$, a dla całej procedury APSP – $O(nm \log n)$. W szczególności dla grafów gęstych ($m = \Theta(n^2)$) daje to $O(n^3 \log n)$, czyli wynik gorszy niż złożoność algorytmu Floyda–Warshalla.

Rozdział 3

Algorytm Seidela

Optymalizacja rozwiązania problemu najkrótszych ścieżek dla wszystkich par wierzchołków w grafach nieskierowanych i nieważonych została zaproponowana przez R. Seidela [4] w 1995 roku. W tej części zaprezentujemy i poddamy analizie główną funkcję jego algorytmu, oznaczaną jako APD (ang. *All-Pairs Distances*), która wyznacza rekurencyjnie macierz odległości D .

Intuicja stojąca za algorytmem opiera się na rekurencyjnym wykorzystaniu mnożenia macierzy do redukcji problemu. Kluczowa obserwacja jest następująca: jeśli zdefiniujemy nowy graf G' , w którym dwa wierzchołki są połączone krawędzią dokładnie wtedy, gdy w oryginalnym grafie G ich odległość wynosi co najwyżej 2, to średnica G' jest dwukrotnie mniejsza niż średnica G . Macierz sąsiedztwa G' można uzyskać jednym mnożeniem macierzy ($A \cdot A$ wykrywa pary połączone ścieżką długości 2), a następnie rekurencyjnie rozwiązać problem w mniejszym grafie. Ponieważ każdy poziom rekurencji zmniejsza średnicę o połowę, to jej maksymalna głębokość wynosi $O(\log n)$.

Algorytm 3 APD – All-Pairs Distances w grafie nieważonym nieskierowanym

Wejście: A : macierz sąsiedztwa $n \times n$ grafu G (macierz 0-1)

Wyjście: D : macierz odległości $n \times n$

```
1:  $Z \leftarrow A \cdot A$ 
2:  $B \leftarrow [i \neq j \wedge (a_{ij} = 1 \vee z_{ij} > 0)]_{i,j=1,\dots,n}$ 
3: if  $b_{ij} = 1$  dla wszystkich  $i \neq j$  then
4:    $D \leftarrow 2B - A$ 
5:   return  $D$ 
6: end if
7:  $T \leftarrow \text{SEIDEL}(B)$ 
8:  $X \leftarrow T \cdot A$ 
9:  $D \leftarrow [2t_{ij} - [x_{ij} < t_{ij} \cdot \deg(j)]]_{i,j=1,\dots,n}$ 
10: return  $D$ 
```

3.1 Poprawność algorytmu

Analizę algorytmu rozpoczniemy od kilku kluczowych obserwacji, formułując je w postaci lematów.

Lemat 3.1.1. *Niech $Z = A \cdot A$. Istnieje ścieżka długości 2 w grafie G pomiędzy wierzchołkami i oraz j wtedy i tylko wtedy, gdy $z_{ij} > 0$.*

Wynika to bezpośrednio z definicji mnożenia macierzy: wpis z_{ij} zlicza liczbę wierzchołków k , które są wspólnymi sąsiadami i oraz j .

Niech G' będzie prostym, nieskierowanym grafem, którego macierzą sąsiedztwa jest wyliczona w algorytmie macierz B . W grafie G' wierzchołki i oraz j są połączone krawędzią wtedy i tylko wtedy, gdy w oryginalnym grafie G istnieje między nimi ścieżka długości co najwyżej 2. Zauważmy, że jeśli graf G' byłby grafem pełnym (co sprawdzamy w kroku 3 algorytmu), oznaczałoby to, że średnica oryginalnego grafu G wynosi maksymalnie 2. W takiej sytuacji macierz odległości D spełniałaby zależność: jeśli $a_{ij} = 0$, to $d_{ij} = 2$, a jeśli $a_{ij} = 1$, to $d_{ij} = 1$. Zależność $D = 2B - A$ z kroku 3 odwzorowuje ten fakt, co stanowi poprawną bazę rekurencji.

Oznaczmy przez t_{ij} długość najkrótszej ścieżki łączącej i oraz j w nowym grafie G' .

Lemat 3.1.2. *Dla dowolnej pary wierzchołków i, j , parzyste d_{ij} implikuje $d_{ij} = 2t_{ij}$, a nieparzyste d_{ij} implikuje $d_{ij} = 2t_{ij} - 1$.*

Dowód. Jeśli dla pary wierzchołków i, j odległość w G wynosi $d_{ij} = 2s$ i najkrótsza ścieżka to $i = i_0, i_1, \dots, i_{2s-1}, i_{2s} = j$, to z definicji grafu G' ciąg $i = i_0, i_2, i_4, \dots, i_{2s-2}, i_{2s} = j$ stanowi najkrótszą ścieżkę pomiędzy i oraz j w G' i ma długość dokładnie s . Podobnie, jeśli odległość jest nieparzysta, $d_{ij} = 2s - 1$, a ścieżka w G to $i = i_0, i_1, \dots, i_{2s-3}, i_{2s-2}, i_{2s-1} = j$, to w grafie G' ciąg $i = i_0, i_2, i_4, \dots, i_{2s-4}, i_{2s-2}, i_{2s-1} = j$ jest najkrótszą ścieżką o długości s . $\square$

Jeśli zatem rekurencyjnie policzymy wartości t_{ij} , jedyne, co algorytm musi zrobić, to zdeterminować parzystość oryginalnego dystansu d_{ij} , aby wywnioskować jego dokładną wartość. Celowi temu służą dwie kolejne obserwacje.

Lemat 3.1.3. *Niech i oraz j będą parą różnych wierzchołków w G . Dla dowolnego sąsiada k wężła j w grafie G zachodzi $d_{ij} - 1 \leq d_{ik} \leq d_{ij} + 1$. Ponadto, istnieje co najmniej jeden sąsiad k wierzchołka j , dla którego $d_{ik} = d_{ij} - 1$.*

Lemat 3.1.4. *Niech i oraz j będą parą różnych wierzchołków w G . Wtedy:*

1. *Parzyste d_{ij} implikuje $t_{ik} \geq t_{ij}$ dla wszystkich sąsiadów k wierzchołka j w G .*
2. *Nieparzyste d_{ij} implikuje $t_{ik} \leq t_{ij}$ dla wszystkich sąsiadów k wierzchołka j w G , oraz $t_{ik} < t_{ij}$ dla co najmniej jednego sąsiada k wierzchołka j w G .*

Dowód. Załóżmy najpierw, że odległość $d_{ij} = 2s$. Z poprzedniego lematu wiemy, że $d_{ik} \geq 2s - 1$ dla dowolnego sąsiada k wierzchołka j . Jeśli d_{ik} jest parzyste, to $d_{ik} = 2t_{ik} \geq 2s$, skąd $t_{ik} \geq s$. Jeśli d_{ik} jest nieparzyste, to $d_{ik} = 2t_{ik} - 1 \geq 2s - 1$, co również implikuje $2t_{ik} \geq 2s$, a więc $t_{ik} \geq s$. Ponieważ dla parzystego d_{ij} zachodzi $s = t_{ij}$, otrzymujemy $t_{ik} \geq t_{ij}$.

Założmy teraz, że odległość $d_{ij} = 2s - 1$ jest nieparzysta. Z poprzedniego lematu mamy $d_{ik} \leq 2s$ dla każdego sąsiada k . Analizując przypadki podobnie jak wyżej, otrzymujemy, że niezależnie od parzystości d_{ik} zawsze zachodzi $t_{ik} \leq s = t_{ij}$. Co więcej, na mocy poprzedniego lematu istnieje sąsiad k dokładnie na najkrótszej ścieżce, dla którego $d_{ik} = 2s - 2$ (wartość parzysta). Wówczas dla tego konkretnego sąsiada zachodzi $2t_{ik} = 2s - 2$, co daje $t_{ik} = s - 1$. Stąd jasno wynika, że $t_{ik} < s = t_{ij}$. $\square$

To wprost dowodzi poprawności warunku w kroku 7 algorytmu APD. Wartość x_{ij} obliczona jako $(T \cdot A)_{ij}$ jest sumą $\sum_k t_{ik}$ dla wszystkich sąsiadów j . Analizując nierówności z powyższego lematu widzimy, że średnia odległość sąsiadów w grafie G' decyduje o parzystości oryginalnej odległości d_{ij} .

3.2 Analiza złożoności

Oznaczmy przez $f(n, \rho)$ czas działania algorytmu APD dla grafu o n wierzchołkach i średnicy ρ . Z uwagi na to, że graf G' ma średnicę równą co najwyżej $\lceil \rho/2 \rceil$, rekurencja zatrzymuje się dla $\rho \leq 2$.

Czas wykonywania pojedynczego wywołania algorytmu (bez kosztu wywołań rekurencyjnych) jest zdominowany przez dwa mnożenia macierzy: $A \cdot A$ w kroku 1 oraz $T \cdot A$ w kroku 6. Oznaczmy czas potrzebny na pomnożenie dwóch macierzy rozmiaru $n \times n$ przez $Mul(n)$. Pozostałe operacje zajmują czas $O(n^2)$. Otrzymujemy następujące równanie rekurencyjne:

$$f(n, \rho) = \begin{cases} Mul(n) + O(n^2) & \text{dla } \rho \leq 2, \\ 2Mul(n) + O(n^2) + f(n, \lceil \rho/2 \rceil) & \text{dla } \rho > 2. \end{cases}$$

Rozwijając to równanie dla głębokości rekurencji wynoszącej $\lceil \log_2 \rho \rceil$, uzyskujemy:

$$f(n, \rho) = (2\lceil \log_2 \rho \rceil - 1) \cdot Mul(n) + O(n^2 \log \rho).$$

Ponieważ maksymalna średnica ρ w grafie spójnym jest zawsze mniejsza niż n , a koszt standardowego mnożenia macierzy spełnia $Mul(n) = \Omega(n^2)$, drugi wyraz jest asymptotycznie zdominowany przez człon pierwszy. Całkowity czas działania algorytmu APD wynosi zatem $O(Mul(n) \log n)$. Przyjmując $Mul(n) = O(n^\omega)$, otrzymujemy złożoność $O(n^{2,372} \log n)$.

Rozdział 4

Algorytm Shoshana–Zwicka

Algorytm Shoshana–Zwicka [5] rozwiązuje problem wszystkich najkrótszych ścieżek w grafach nieskierowanych z wagami krawędzi ze zbioru $\{1, 2, \dots, M\}$. Algorytm ten działa w czasie $O(Mn^\omega)$. Zaczniemy od omówienia prostszej wersji algorytmu, zwanej USP (ang. *Unweighted Shortest Paths*), przeznaczonej dla grafów nieważonych – zrozumienie tej części jest kluczowe dla wyprowadzenia wariantu ważonego. Następnie uogólnimy ją do algorytmu WUSP (ang. *Weighted Undirected Shortest Paths*) dla grafów ważonych. Na końcu omówimy błąd w oryginalnym sformułowaniu algorytmu, wskazany przez Eirinakisa, Williamsona i Subramaniego [6].

4.1 Definicje

4.1.1 Iloczyn i macierze indykatorowe

Definicja 4.1.1 ([5], Iloczyn odległościowy). *Dla dwóch macierzy $A = (a_{ij})$ oraz $B = (b_{ij})$ rozmiaru $n \times n$ ich iloczynem odległościowym (iloczynem $(\min, +)$) nazywamy macierz $C = A \star B$ wymiaru $n \times n$ o elementach*

$$c_{ij} = \min_{k=1, \dots, n} \{a_{ik} + b_{kj}\}, \quad 1 \leq i, j \leq n.$$

Definicja 4.1.2 (Macierz wskaźnikowa – wersja boolowska). *Dla zbioru $X \subseteq \{0, 1, \dots, n\}$ macierz indykatorową D_X nazywamy macierz $n \times n$ zadaną wzorem*

$$(D_X)_{ij} = \begin{cases} 1, & \text{gdy } \delta(i, j) \in X, \\ 0, & \text{w przeciwnym razie.} \end{cases}$$

Definicja 4.1.3 (Operacje na zbiorach odległości). *Dla dwóch zbiorów odległości X, Y definiujemy:*

$$X + Y = \{x + y : x \in X, y \in Y\}, \quad X \bullet Y = \bigcup_{x \in X, y \in Y} [|x - y|, x + y].$$

Definicja 4.1.4 (Separacja zbioru). *Niech $X = \bigcup_{r=1}^p [a_r, b_r]$, gdzie $a_r \leq b_r$ dla $1 \leq r \leq p$ oraz $b_r < a_{r+1}$ dla $1 \leq r < p$. Separację zbioru X definiujemy jako*

$$\text{sep}(X) = \min\{a_r - b_{r-1} : r = 2, \dots, p\}.$$

Definicja 4.1.5 (Macierz indykatorowa – wersja ważona). Dla zbioru $X = \bigcup_{r=1}^p [a_r, b_r] \subseteq [0, Mn]$ macierzą D_X nazywamy macierz $n \times n$ o elementach

$$(D_X)_{ij} = \begin{cases} -M & \text{gdy } a_r \leq \delta(i, j) \leq b_r - M \text{ dla pewnego } r \text{ (strefa przycięta),} \\ \delta(i, j) - b_r & \text{gdy } b_r - M < \delta(i, j) \leq b_r + M \text{ (strefa dokładna),} \\ +\infty & \text{w przeciwnym razie (strefa nieskończona).} \end{cases}$$

4.1.2 Operacje liczbowe na macierzach

Definicja 4.1.6 (Obcinanie). Dla macierzy $A = (a_{ij})$ oraz $a \leq b$ definiujemy macierz $\text{clip}(A, a, b)$ o elementach

$$(\text{clip}(A, a, b))_{ij} = \begin{cases} a & \text{gdy } a_{ij} < a, \\ a_{ij} & \text{gdy } a \leq a_{ij} \leq b, \\ +\infty & \text{gdy } a_{ij} > b. \end{cases}$$

Definicja 4.1.7 (Wycinanie). Dla macierzy $A = (a_{ij})$ oraz $a \leq b$ definiujemy macierz $\text{chop}(A, a, b)$ o elementach

$$(\text{chop}(A, a, b))_{ij} = \begin{cases} a_{ij} & \text{gdy } a \leq a_{ij} \leq b, \\ +\infty & \text{w przeciwnym razie.} \end{cases}$$

Definicja 4.1.8 (Liczbowe odpowiedniki operacji boolowskich). Dla macierzy $A = (a_{ij})$ oraz $B = (b_{ij})$ definiujemy:

$$\begin{aligned} (A \hat{\wedge} B)_{ij} &= \begin{cases} a_{ij} & \text{gdy } b_{ij} < 0, \\ +\infty & \text{w przeciwnym razie,} \end{cases} \\ (A \hat{\wedge} \neg B)_{ij} &= \begin{cases} a_{ij} & \text{gdy } b_{ij} \geq 0, \\ +\infty & \text{w przeciwnym razie,} \end{cases} \\ (A \hat{\vee} B)_{ij} &= \begin{cases} a_{ij} & \text{gdy } a_{ij} \neq +\infty, \\ b_{ij} & \text{gdy } a_{ij} = +\infty, b_{ij} \neq +\infty, \\ +\infty & \text{gdy } a_{ij} = b_{ij} = +\infty. \end{cases} \end{aligned}$$

Są to liczbowe odpowiedniki boolowskich $\wedge$, $\wedge \neg$, $\vee$.

4.2 Wprowadzenie teoretyczne

Głównym narzędziem algorytmu jest iloczyn odległościowy macierzy oznaczany jako iloczyn $(\min, +)$. Iloczyn odległościowy dwóch macierzy $n \times n$ o elementach ze zbioru $\{1, 2, \dots, M, +\infty\}$ można obliczyć w czasie $O(Mn^\omega)$ przy użyciu algorytmu Alona, Galila i Margalita [17].

Obliczanie iloczynów odległościowych efektywnie jest jednak nietrywialnym zadaniem. Przypomnijmy, że dla dwóch macierzy $A = (a_{ij})$ i $B = (b_{ij})$ rozmiaru $n \times n$ ich iloczyn odległościowy definiujemy jako macierz $C = A \star B$ o elementach

$$c_{ij} = \min_{k=1, \dots, n} \{a_{ik} + b_{kj}\}.$$

Iloczyn ten wydaje się być podobny do zwykłego mnożenia macierzy, w którym $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$: w obu przypadkach element (i, j) wyznaczamy przez kombinację i -tego wiersza A z j -tą kolumną B . Różnica polega jednak na zastąpieniu pary operacji $(\sum, \cdot)$ parą $(\min, +)$.

Struktura algebraiczna $(\min, +)$ tworzy *półpierścień*: działanie $\min$ pełni rolę dodawania, a $+$ pełni rolę mnożenia. Półpierścień różni się od pierścienia brakiem elementu odwrotnego dla $\min$ – właśnie ta różnica jest kluczowa. Kerr [18] wykazał, że w modelu algebraicznym dopuszczającym wyłącznie operacje $\min$ i $+$ obliczenie iloczynu odległościowego wymaga $\Omega(n^3)$ operacji. Oznacza to, że algorytm szybkie mnożenia macierzy w czasie subkubicznym [14] nie może być zastosowany bezpośrednio do iloczynu odległościowego – algorytmy te zakładają istnienie struktury pierścienia, której półpierścień $(\min, +)$ nie spełnia.

4.2.1 Redukcja do mnożeń boolowskich

Alon, Galil i Margalit [17] pokazali, jak ominąć ograniczenie Kerra przez redukcję iloczynu odległościowego do skończonej liczby zwykłych mnożeń macierzy boolowskich. Mnożenie boolowskie macierzy obliczamy przez zwykłe mnożenie macierzy nad $\mathbb{Z}$ (następnie zamieniając każdy wpis niezerowy na 1), co kosztuje $O(n^\omega)$.

Redukcja opiera się na następującej obserwacji. Niech A i B będą macierzami o elementach całkowitoliczbowych z $\{1, \dots, M\}$. Dla każdego progu $t \in \{1, \dots, M\}$ definiujemy macierze progowe:

$$A_{ij}^{(t)} = \mathbf{1}[a_{ij} \leq t], \quad B_{ij}^{(t)} = \mathbf{1}[b_{ij} \leq t].$$

Lemat 4.2.1 ([17, Twierdzenie 3]). *Niech A i B będą macierzami $n \times n$ o elementach całkowitoliczbowych z $\{1, \dots, M\}$, a $C = A \star B$ ich iloczynem odległościowym. Wówczas dla każdej pary (i, j) :*

$$c_{ij} = \min \left\{ s \in \{2, \dots, 2M\} : \exists t \in \{1, \dots, s-1\} \text{ takie, że } (A^{(t)} \cdot B^{(s-t)})_{ij} = 1 \right\}.$$

Dowód. Ustalmy parę (i, j) i niech $s = c_{ij}$, to znaczy $s = \min_k \{a_{ik} + b_{kj}\}$. Niech k^* będzie indeksem realizującym to minimum. Niech $t = a_{ik^*}$, wówczas $b_{k^*j} = s - t$. Z definicji macierzy progowych mamy $A_{ik^*}^{(t)} = \mathbf{1}[a_{ik^*} \leq t] = 1$ oraz $B_{k^*j}^{(s-t)} = \mathbf{1}[b_{k^*j} \leq s - t] = 1$, skąd $(A^{(t)} \cdot B^{(s-t)})_{ij} = 1$ dla $t = a_{ik^*}$.

Z drugiej strony, jeśli $(A^{(t)} \cdot B^{(s'-t)})_{ij} = 1$ dla pewnych t i $s' < s$, to istnieje k takie, że $a_{ik} \leq t$ i $b_{kj} \leq s' - t$, skąd $a_{ik} + b_{kj} \leq s' < s$ – sprzeczność z definicją $s = \min_k \{a_{ik} + b_{kj}\}$. Zatem s jest najmniejszą wartością, dla której iloczyn boolowski daje jedynkę, co kończy dowód. $\square$

4.2.2 Złożoność obliczania iloczynu odległościowego

Na podstawie Lematu 4.2.1 możemy obliczyć c_{ij} dla wszystkich par (i, j) jednocześnie, iterując $s = 2, 3, \dots, 2M$ i dla każdego s sprawdzając, dla których par iloczyn boolowski $A^{(t)} \cdot B^{(s-t)}$ po raz pierwszy daje jedynkę. Łączna liczba mnożeń boolowskich wynosi $O(M)$ (dla każdego s wystarczy jedno mnożenie z odpowiednio dobranym t), a każde z nich kosztuje $O(n^\omega)$. Stąd:

Twierdzenie 4.2.2 ([17, Twierdzenie 3]). *Iloczyn odległościowy dwóch macierzy $n \times n$ o elementach całkowitoliczbowych z $\{1, \dots, M\}$ można obliczyć w czasie $O(Mn^\omega)$.*

$$\begin{array}{c}
\underbrace{\text{Iloczyn } (\min, +)} \\
\Omega(n^3) \text{ bez redukcji} \\
\downarrow \text{Lemat 4.2.1} \\
\underbrace{O(M) \text{ mnożeń boolowskich}} \\
O(n^\omega) \text{ każde} \\
\downarrow \text{redukcja} \\
\underbrace{O(M) \text{ mnożeń nad } \mathbb{Z}} \\
O(n^\omega) \text{ każde} \\
\downarrow \\
O(Mn^\omega)
\end{array}$$

4.2.3 Podejście naiwne i jego ograniczenia

Jeśli D jest macierzą sąsiedztwa grafu ważonego (d_{ij} to waga krawędzi (i, j) lub $+\infty$ w przypadku braku krawędzi, a $d_{ii} = 0$), to macierz D^n zawiera wszystkie najkrótsze odległości w grafie. Macierz tę można obliczyć za pomocą $\lceil \log_2 n \rceil$ mnożeń metodą szybkiego potęgowania.

Naiwne podejście ma taką wadę, że nawet gdy wyjściowe wagi są ograniczone przez M , to elementy kolejnych potęg mogą sięgać nM , a obliczanie iloczynów odległościowych macierzy o tak dużych elementach jest drogie. Główny wkład pracy Shoshana i Zwicka polega na pokazaniu, że problem APSP dla grafów nieskierowanych można rozwiązać przy pomocy jedynie $O(\log(Mn))$ iloczynów.

Kluczową własnością grafów nieskierowanych, wykorzystywaną w algorytmie, jest obustronna nierówność trójkąta.

Lemat 4.2.3. *Dla dowolnych trzech wierzchołków $i, j, k \in V$ w grafie nieskierowanym zachodzi:*

$$|\delta(i, k) - \delta(k, j)| \leq \delta(i, j) \leq \delta(i, k) + \delta(k, j).$$

Nierówność lewostronna, to odwrotna nierówność trójkąta i wynika z faktu, że $\delta(j, k) = \delta(k, j)$ w grafach nieskierowanych. Właśnie ta własność umożliwia utrzymanie elementów macierzy w ograniczonym zakresie.

4.3 Algorytm USP dla grafów nieważonych

Rozważmy najpierw przypadek grafu nieważonego $G = (V, E)$ z $|V| = n$, którego macierz sąsiedztwa oznaczamy przez A . Zakładając spójność grafu, wszystkie odległości spełniają $0 \leq \delta(i, j) < n$, więc do ich zakodowania wystarczy $\ell = \lceil \log_2 n \rceil$ bitów.

Zamiast wyznaczać odległości jako liczbę, algorytm znajduje jej kodowanie bit po bicie. Dla każdej pozycji k algorytm odpowiada na pytanie boolowskie „czy k -ty bit odległości $\delta(i, j)$ jest ustawiony?”. Aby odpowiedzieć na to pytanie, wystarczy sprawdzić, czy $\delta(i, j) \bmod 2^{k+1} \geq 2^k$ – czyli czy odległość leży w prawej połowie bloku rozmiaru 2^{k+1} .

Lemat 4.3.1. *Dla dowolnych zbiorów odległości X, Y oraz dowolnego grafu nieskierowanego dla wszystkich $i, j = 1, \dots, n$ zachodzi:*

$$(D_{X+Y})_{ij} \leq (D_X \cdot D_Y)_{ij} \leq (D_{X \bullet Y})_{ij},$$

dla $\cdot$ oznaczającego iloczyn macierzy boolowskich.

Dowód. Dla pierwszej nierówności założymy, że $(D_{X+Y})_{ij} = 1$, tj. $\delta(i, j) = x + y$ dla pewnych $x \in X, y \in Y$. Niech p będzie najkrótszą ścieżką z i do j , a k – wierzchołkiem na p spełniającym $\delta(i, k) = x$; wówczas $\delta(k, j) = y$, więc iloczyn w pozycji (i, j) daje 1. Dla drugiej nierówności założymy, że iloczyn daje 1 w pozycji (i, j) , tj. istnieje k z $\delta(i, k) \in X$ oraz $\delta(k, j) \in Y$; z nierówności trójkąta $\delta(i, j) \in X \bullet Y$. $\square$

Lemat ten jest istotny dla całego algorytmu, ponieważ ustala związek między mnożeniem boolowskim (które jest szybkie – $O(n^\omega)$) a informacją o odległościach, którą liczymy. Bezpośrednie obliczenie odległości wymaga iloczynu $(\min, +)$ o złożoności $\Omega(n^3)$. Mnożenie boolowskie jest znacznie tańsze, ale nie oblicza odległości wprost – odpowiada jedynie na pytanie o istnienie świadka k spełniającego $\delta(i, k) \in X$ i $\delta(k, j) \in Y$.

Dolne ograniczenie $D_{X+Y} \leq D_X \cdot D_Y$ gwarantuje, że żadne prawdziwe trafienie nie zostanie utracone: jeśli $\delta(i, j) = x + y$ dla $x \in X, y \in Y$, to na najkrótszej ścieżce z i do j istnieje wierzchołek k z $\delta(i, k) = x$ i $\delta(k, j) = y$, więc mnożenie boolowskie na pewno zwróci jedynkę.

Górne ograniczenie $D_X \cdot D_Y \leq D_{X \bullet Y}$ ogranicza zakres fałszywych trafień: mnożenie boolowskie może dać jedynkę nawet gdy $\delta(i, j) \notin X + Y$, ponieważ świadek k z $\delta(i, k) \in X$ i $\delta(k, j) \in Y$ nie musi leżeć na najkrótszej ścieżce z i do j . Z obustronnej nierówności trójkąta wynika jedynie $\delta(i, j) \in X \bullet Y$, co jest zbiorem szerszym niż $X + Y$.

Cały algorytm USP jest zbudowany wokół tego, że mnożenie boolowskie jest szybkie i służy jako niedokładne narzędzie do wyznaczania bitów odległości, a krok filtrowania $(\wedge C_{k+1})$ eliminuje fałszywe trafienia, korzystając z informacji o bitach wyznaczonych we wcześniejszych iteracjach. Bez lematu nie wiedzielibyśmy, czy fałszywe trafienia są w ogóle kontrolowalne – lemat gwarantuje, że leżą one w przewidywalnym zbiorze $X \bullet Y \setminus (X + Y)$.

Algorytm USP, przedstawiony jako Algorytm 4, składa się z trzech faz. W pierwszej fazie obliczane są macierze indykatorowe $A_k = D_{[0, 2^k)}$ oraz $B_k = D_{[0, 2^k]}$, tj. macierze 0–1, w których wpis (i, j) wynosi 1 dokładnie wtedy, gdy odległość $\delta(i, j)$ należy do odpowiedniego przedziału. Innymi słowy, A_k wskazuje pary wierzchołków odległe o mniej niż 2^k , a B_k – o co najwyżej 2^k . Macierze te są obliczane przez potęgowanie kwadratowe: wykorzystując tożsamość $[0, 2^k] \subseteq [0, 2^{k-1}] \bullet [0, 2^{k-1}]$, otrzymujemy A_k i B_k jako iloczyny boolowskie macierzy z poprzedniej iteracji. W drugiej fazie, schodząc od bitu najwyższego ku najniższemu, wyznaczane są macierze C_k kodujące bity odległości. Trzecia faza składa bity w pełną macierz odległości.

Trzonem algorytmu jest pętla schodząca z $k = \ell - 1, \ell - 2, \dots, 0$, obliczająca macierze C_k . Macierze te mają konkretną interpretację: $(C_k)_{ij} = 1$ dokładnie wtedy, gdy k -ty bit $\delta(i, j)$ wynosi zero. To bezpośrednio daje końcowy wzór $D = \sum_{k=0}^{\ell} 2^k (\neg C_k)$.

Aby zrozumieć, skąd bierze się formuła obliczająca C_k , wyobraźmy sobie oś liczbową podzieloną na bloki rozmiaru 2^{k+2} (czyli dwa razy większe niż bloki istotne dla C_k). Każdy taki duży blok zawiera cztery ćwiartki rozmiaru 2^k . Macierze z poprzedniej iteracji kodują następujące informacje: C_{k+1} rozróżnia, czy odległość leży w lewej czy prawej połowie dużego bloku; P_{k+1} wskazuje pary, których odległość jest na lewym końcu dużego bloku, a Q_{k+1} – na środku.

Lemat 4.3.2. *Dla każdego $1 \leq k \leq \ell$ po zakończeniu pętli zachodzi:*

$$C_k = D_{\{x : 0 \leq x \bmod 2^{k+1} < 2^k\}}, \quad F_k = D_{\{x : 0 \leq x \bmod 2^{k+1} \leq 2^k\}}$$

$$P_k = D_{\{x : x \bmod 2^{k+1} = 0\}}, \quad Q_k = D_{\{x : x \bmod 2^{k+1} = 2^k\}}.$$

Algorytm 4 USP – APSP w grafie nieważonym nieskierowanym

Wejście: A : macierz sąsiedztwa $n \times n$

Wyjście: D : macierz odległości $n \times n$

```
1:  $\ell \leftarrow \lceil \log_2 n \rceil$ 
2:  $A_0 \leftarrow I$ ;  $B_0 \leftarrow A \vee I$ 
3: for  $k = 1, 2, \dots, \ell$  do
4:    $A_k \leftarrow A_{k-1} \cdot B_{k-1}$ 
5:    $B_k \leftarrow B_{k-1} \cdot B_{k-1}$ 
6: end for
7:  $C_\ell \leftarrow \mathbf{1}$ ;  $F_\ell \leftarrow \mathbf{1}$ ;  $P_\ell \leftarrow I$ ;  $Q_\ell \leftarrow \mathbf{0}$ 
8: for  $k = \ell - 1, \ell - 2, \dots, 0$  do
9:    $C_k \leftarrow [(P_{k+1} \cdot A_k) \wedge C_{k+1}] \vee [(Q_{k+1} \cdot A_k) \wedge \neg C_{k+1}]$ 
10:   $F_k \leftarrow [(P_{k+1} \cdot B_k) \wedge C_{k+1}] \vee [(Q_{k+1} \cdot B_k) \wedge \neg C_{k+1}]$ 
11:   $P_k \leftarrow P_{k+1} \vee Q_{k+1}$ 
12:   $Q_k \leftarrow F_k \wedge \neg C_k$ 
13: end for
14:  $D \leftarrow \sum_{k=0}^{\ell} 2^k (\neg C_k)$ 
15: return  $D$ 
```

Pierwszy przypadek rozpoznaje iloczyn $P_{k+1} \cdot A_k$ – bo szuka par (i, j) , dla których istnieje świadek s z $\delta(i, s) \bmod 2^{k+2}$ bliskim zeru oraz $\delta(s, j) < 2^k$. Prawdziwe trafienia leżą jednak zawsze w lewej połowie większego bloku, a tę sytuację rozpoznaje C_{k+1} . Złożenie z $\wedge C_{k+1}$ eliminuje zatem fałszywe trafienia. Drugi przypadek obsługujemy analogicznie, tylko zamieniamy P_{k+1} na Q_{k+1} i filtr zmieniamy na $\wedge \neg C_{k+1}$. Pozostałe aktualizacje wynikają z prostszych tożsamości – zbiór lewych krańców mniejszych bloków to suma lewych krańców i środków większych bloków (stąd $P_k = P_{k+1} \vee Q_{k+1}$), a środki mniejszych bloków to dokładnie te odległości, które należą do F_k , ale nie do C_k (stąd $Q_k = F_k \wedge \neg C_k$).

Twierdzenie 4.3.3. *Algorytm USP oblicza wszystkie odległości w grafie nieważonym nieskierowanym przy użyciu $O(\log n)$ iloczynów macierzy boolowskich, a jego łączny czas działania wynosi $O(n^\omega)$.*

4.4 Algorytm WUSP dla grafów ważonych

Uogólnienie algorytmu USP na grafy ważne napotyka na fundamentalny problem: w grafie nieważonym odległości leżą w zakresie $[0, n)$ i do ich zakodowania wystarczy $\ell = \lceil \log_2 n \rceil$ bitów, ale w grafie z wagami ze zbioru $\{1, \dots, M\}$ odległości sięgają $(n-1)M$, a iloczyn odległościowy macierzy o tak dużych elementach ma złożoność $O(nM \cdot n^\omega)$.

Główna idea Shoshana i Zwicka polega na tym, żeby nigdy nie przechowywać pełnych odległości w macierzach. Zamiast tego, każda macierz "widzi" odległości przez wąskie okno szerokości $2M$, przechowując jedynie przesunięcie $\delta(i, j) - b_r$ względem najbliższego punktu odniesienia b_r . Dzięki temu elementy macierzy pozostają w zakresie $[-M, M] \cup \{+\infty\}$, a każdy iloczyn odległościowy ma złożoność $O(Mn^\omega)$.

Dlaczego okienko szerokości $2M$ wystarcza? Kluczową rolę odgrywa następujący fakt, wynikający z ograniczenia wag krawędzi:

Lemat 4.4.1 ([5], Lemat okienkowy). *Niech p będzie ścieżką z i do j w grafie z wagami krawędzi z $[1, M]$. Jeśli x i y są kolejnymi wierzchołkami na p , to odległość od j zmienia się o co najwyżej M , tj. $|p(j, y) - p(j, x)| \leq M$.*

Oznacza to, że na każdej najkrótszej ścieżce zawsze istnieje wierzchołek, którego odległość od źródła wpada w dowolne okno o szerokości M . To gwarantuje, że przy obliczaniu iloczynu odległościowego świadek k minimalizujący $a_{ik} + b_{kj}$ zawsze znajdzie się w strefie dokładnej okienka – nigdy nie utracimy informacji potrzebnej do poprawnego obliczenia.

Omówmy dokładnie przejście z wersji nieważonej (USP) do ważonej (WUSP). Algorytm USP operuje na bitach odległości bezpośrednio: k -ty bit odległości $\delta(i, j)$ jest ustawiony, gdy $\delta(i, j) \bmod 2^{k+1} \geq 2^k$. W WUSP schemat jest taki sam, ale przeskalowany o czynnik M : zamiast bloków rozmiaru 2^k mamy bloki rozmiaru 2^{k+m} (bo $M = 2^m$), a zamiast macierzy boolowskich 0–1 mamy macierze o wartościach w $[-M, M]$. Tabela poniżej podsumowuje te zależności:

	USP (nieważony)	WUSP (ważony)
Zakres odległości	$[0, n - 1]$	$[0, (n - 1)M]$
Rozmiar bloku	2^k	2^{k+m}
Typ macierzy	boolowskie $\{0, 1\}$	liczbowe $[-M, M] \cup \{+\infty\}$
Iloczyn	boolowski $\cdot$	odległościowy $\star$ i clip
Operacje logiczne	$\wedge, \wedge \neg, \vee$	$\hat{\wedge}, \hat{\wedge} \neg, \hat{\vee}$
Wynik bitu k	$(C_k)_{ij} = 0$	$(C_k)_{ij} \geq 0$

W WUSP iloczyn odległościowy $D_X \star D_Y$ może wygenerować wartości wykraczające poza $[-M, M]$. Operacja $\text{clip}(\cdot, -M, M)$ przycina te wartości – z lematu 4.4.2 o iloczynach indyktorowych wynika, że takie przycięcie nie traci informacji potrzebnej do odróżnienia prawdziwych trafień od fałszywych.

Lemat 4.4.2. *Niech X i $Y = [0, y]$ będą zbiorami odległości, przy czym $y \geq M$ oraz $\text{sep}(X) > 2y + 3M$. Wówczas:*

$$D_{X \bullet Y} \geq \text{clip}(D_X \star D_Y, -M, M) \geq D_{X+Y}.$$

Lemat ten jest ważoną wersją lematu z algorytmu USP. Dowód rozpada się na cztery przypadki w zależności od stref (przycięta, dokładna, nieskończona), w których leżą elementy $(D_X)_{ik}$ oraz $(D_Y)_{kj}$ dla świadka k , i w każdym z nich wykorzystuje lemat okienkowy do zagwarantowania istnienia odpowiedniego świadka.

Warunek $\text{sep}(X) > 2y + 3M$ gwarantuje, że okna dokładności sąsiednich przedziałów zbioru X nie nakładają się, nawet po uwzględnieniu zbioru Y . Dzięki temu błędne trafienia są eliminowane przez clip.

Algorytm WUSP, przedstawiony jako Algorytm 5, składa się z czterech etapów.

Etap 1 (linie 2–4): potęgowanie macierzy wag w iloczynie odległościowym z obcinaniem wartości powyżej $2M$. Po $m + 1$ iteracjach macierz D zawiera dokładne odległości dla par o $\delta(i, j) \leq 2M$ i $+\infty$ dla pozostałych. Ten etap nie ma odpowiednika w USP, ale jest niezbędny, ponieważ w grafie ważonym sąsiedzi mogą być oddaleni o więcej niż 1, a algorytm musi znać bliskie odległości, aby poprawnie zainicjalizować kolejne etapy.

Etap 2 (linie 6–8): budowa macierzy $A_0, A_1, \dots, A_\ell$, gdzie $A_k = D_{[0, 2^{k+m}]}$. Każda macierz A_k koduje, czy odległość $\delta(i, j)$ wynosi co najwyżej 2^{k+m} , ale nie przez wartość boolowską, lecz przez przesunięcie w okienku – dokładnie tak, jak B_k w USP kodowała $\delta(i, j) \leq 2^k$, ale teraz z dokładnością do M . Krok indukcyjny wynika z lematu 4.4.2 zastosowanego do $X = Y = [0, 2^{k+m-1}]$ i tożsamości $[0, 2^a] \bullet [0, 2^a] = [0, 2^{a+1}]$.

Algorytm 5 WUSP – APSP w grafie ważonym nieskierowanym

Wejście: A : macierz wag $n \times n$ (wartości w $\{1, \dots, M\} \cup \{+\infty\}$), $M = 2^m$

Wyjście: D : macierz odległości

```

1:  $\ell \leftarrow \lceil \log_2 n \rceil$ ;  $m \leftarrow \log_2 M$ 
2: for  $k = 1$  do  $m + 1$  do
3:    $A \leftarrow \text{clip}(A \star A, 0, 2M)$ 
4: end for
5:  $A_0 \leftarrow A - M$ 
6: for  $k = 1$  do  $\ell$  do
7:    $A_k \leftarrow \text{clip}(A_{k-1} \star A_{k-1}, -M, M)$ 
8: end for
9:  $C_\ell \leftarrow -M$ ;  $P_\ell \leftarrow \text{clip}(A, 0, M)$ ;  $Q_\ell \leftarrow +\infty$ 
10: for  $k = \ell - 1$  w dół do  $0$  do
11:    $C_k \leftarrow [\text{clip}(P_{k+1} \star A_k, -M, M) \hat{\wedge} C_{k+1}] \hat{\vee} [\text{clip}(Q_{k+1} \star A_k, -M, M) \hat{\wedge} \neg C_{k+1}]$ 
12:    $P_k \leftarrow P_{k+1} \hat{\vee} Q_{k+1}$ 
13:    $Q_k \leftarrow \text{chop}(C_k, 1 - M, M)$ 
14: end for
15: for  $k = 1$  do  $\ell$  do
16:    $B_k \leftarrow (C_k \geq 0)$ 
17: end for
18:  $(B_0)_{ij} \leftarrow (0 \leq (P_0)_{ij} < M)$ 
19:  $R \leftarrow P_0 \bmod M$ 
20:  $D \leftarrow M \cdot \sum_{k=0}^{\ell} 2^k B_k + R$ 
21: return  $D$ 

```

Etap 3 (linie 10–14): analog głównej pętli USP, operujący na blokach długości przeskalowanej przez M . Zdefiniujmy zbiory

$$\begin{aligned}
X_k &= \{x : 0 \leq x \bmod 2^{k+m+1} \leq 2^{k+m}\}, \\
Y_k &= \{x : x \bmod 2^{k+m+1} = 0\}, \\
Z_k &= \{x : x \bmod 2^{k+m+1} = 2^{k+m}\}.
\end{aligned}$$

Są to dokładne odpowiedniki zbiorów z USP, z zastąpieniem 2^k przez 2^{k+m} . Po zakończeniu pętli zachodzi $C_k = D_{X_k}$, $P_k = D_{Y_k}$, $Q_k = D_{Z_k}$. Mechanizm eliminacji fałszywych trafień jest identyczny jak w USP: operacja $\hat{\wedge} C_{k+1}$ (odpowiednio $\hat{\wedge} \neg C_{k+1}$) filtruje trafienia pochodzące z niewłaściwej połowy bloku – z tą różnicą, że zamiast wartości boolowskiej 0/1 jako filtra używa się znaku wartości C_{k+1} .

Etap 4 (linie 16–20): złożenie bitów odległości. Dla $k \geq 1$ test $(C_k)_{ij} \geq 0$ sprawdza, czy $\delta(i, j)$ leży poza strefą przyciętą X_k , co oznacza, że $(k + m)$ -ty bit jest ustawiony. Bit na pozycji m oraz m najniższych bitów wymagają osobnej obsługi, gdyż macierze C_0 i A_{-1} nie istnieją – oryginalne sformułowanie tego etapu zawiera błąd, który omawiamy w następnym podrozdziale.

Twierdzenie 4.4.3. *Algorytm WUSP oblicza wszystkie odległości w grafie ważonym nieskierowanym o wagach krawędzi w zakresie $\{1, \dots, M\}$ przy użyciu $O(\log(Mn))$ iloczynów odległościowych macierzy o elementach skończonych w zakresie $\{-M, \dots, M\}$. Łączny czas działania wynosi $O(Mn^\omega)$.*

4.5 Korekta algorytmu Shoshana–Zwicka

Eirinakis, Williamson i Subramani [6] wykazali, że oryginalna wersja algorytmu Shoshana–Zwicka zawiera błąd w ostatnim etapie odtwarzania odległości. Sama konstrukcja macierzy A_k , C_k , P_k , Q_k pozostaje poprawna; problem dotyczy wyłącznie sposobu interpretacji macierzy P_0 w końcowym wzorze.

W oryginalnej wersji ostatni etap wyznacza

$$B_0 = (0 \leq P_0 < M), \quad R = P_0 \bmod M, \quad D = M \sum_{k=0}^{\ell} 2^k B_k + R,$$

przy założeniu, że B_0 koduje m -ty bit odległości (o wadze $M = 2^m$), a R przechowuje m najniższych bitów, czyli $\delta(i, j) \bmod M$. Składnik $MB_0 + R$ miał wówczas odtwarzać $\delta(i, j) \bmod 2M$. Milcząco przyjęto, że $(P_0)_{ij} \geq 0$ – a to nie zawsze jest prawdą.

Skąd dokładnie biorą się ujemne wartości P_0 ? Wiemy, że $P_0 = D_{Y_0}$, gdzie $Y_0 = \{0, 2M, 4M, \dots\}$. Każdy punkt $b_r = r \cdot 2M$ wyznacza okno dokładności $(b_r - M, b_r + M]$. Sąsiednie okna stykają się i pokrywają całą oś, a wartość $(P_0)_{ij} = \delta(i, j) - b_r$ zależy od tego, po której stronie najbliższego punktu Y_0 leży $\delta(i, j)$.

Zapisując $\delta(i, j) = q \cdot 2M + s$, gdzie $s = \delta(i, j) \bmod 2M$, otrzymujemy:

$$(P_0)_{ij} = \begin{cases} s, & \text{gdy } s \leq M \quad (\delta \text{ leży w oknie wokół } q \cdot 2M), \\ s - 2M, & \text{gdy } s > M \quad (\delta \text{ leży w oknie wokół } (q+1) \cdot 2M). \end{cases}$$

W pierwszym przypadku $(P_0)_{ij} \in [0, M]$, w drugim $(P_0)_{ij} \in (-M, 0)$.

Wynika z tego, że $(P_0)_{ij} < 0$ dokładnie wtedy, gdy $\delta(i, j) \bmod 2M > M$, czyli gdy m -ty bit odległości jest równy 1. Formuła $B_0 = (0 \leq P_0 < M)$ opisuje sytuację przeciwną, czyli odpowiada negacji m -tego bitu. Ponadto $R = P_0 \bmod M$ nie daje poprawnej wartości $\delta(i, j) \bmod M$, gdy $(P_0)_{ij}$ jest ujemne.

Eirinakis, Williamson i Subramani proponują:

$$\widehat{B}_0 = (-M < P_0 < 0), \quad \widehat{R} = P_0, \quad D = M \sum_{k=1}^{\ell} 2^k B_k + 2M \widehat{B}_0 + \widehat{R}.$$

Idea jest prosta: jeżeli $(P_0)_{ij} \geq 0$, to $(P_0)_{ij}$ to już $\delta \bmod 2M$ i wystarczy je przepisać; jeżeli $(P_0)_{ij} < 0$, to brakuje $2M$, by uzupełnić wartość do $\delta \bmod 2M$. Składnik $2M \widehat{B}_0 + \widehat{R}$ dokładnie to robi, dając

$$(2M \widehat{B}_0 + \widehat{R})_{ij} = \delta(i, j) \bmod 2M.$$

Składnik ten w pełni uwzględnia m -ty bit, dlatego sumowanie B_k zaczyna się od $k = 1$. Korekta zmienia tylko sposób odczytu wyniku i nie wpływa na złożoność algorytmu.

Rozdział 5

Algorytm Aingwortha, Chekuriego, Indyka i Motwaniego

W ostatniej sekcji skupimy się na algorytmie aproksymacyjnym dla problemu najkrótszych ścieżek między wszystkimi parami wierzchołków. Zaprezentujemy wersję w grafach nieskierowanych, nieważonych z pracy Aingwortha, Chekuriego, Indyka i Motwaniego [7]. Algorytm nie korzysta z szybkiego mnożenia macierzy – opiera się wyłącznie na przeszukiwaniu wszerz (BFS) i zbiorach dominujących. Działa w czasie $O(n^{2.5}\sqrt{\log n})$ i zwraca odległości z błędem addytywnym co najwyżej 2.

5.1 Definicje i oznaczenia

Rozważamy graf nieskierowany, nieważony, spójny $G = (V, E)$ z n wierzchołkami i m krawędziami.

Definicja 5.1.1 (Wierzchołki nisko- i wysokostopniowe). *Ustalmy parametr s (próg stopnia). Definiujemy:*

$$\begin{aligned} L(V) &= \{u \in V : d_u < s\} && \text{– wierzchołki niskostopniowe,} \\ H(V) &= V \setminus L(V) = \{u \in V : d_u \geq s\} && \text{– wierzchołki wysokostopniowe.} \end{aligned}$$

Przez $G[L(V)]$ oznaczamy podgraf G indukowany przez $L(V)$.

Definicja 5.1.2 (Zbiór dominujący). *Dla danego zbioru $A \subseteq V$ zbiór $D \subseteq V$ nazywamy zbiorem dominującym dla A , jeśli dla każdego $v \in A$ zachodzi $N(v) \cap D \neq \emptyset$, tzn. każdy wierzchołek z A albo sam należy do D , albo ma sąsiada w D .*

Definicja 5.1.3. *Dla $v \in V$ oznaczamy przez $B(v)$ głębokość drzewa BFS zakorzonego w v , czyli odległość od v do najdalszego osiągalnego wierzchołka.*

Definicja 5.1.4. *Przez $\hat{\delta}(u, v)$ oznaczamy przybliżoną odległość między wierzchołkami u i v zwracaną przez algorytm Approx-APSP.*

5.2 Konstrukcja zbioru dominującego

Twierdzenie 5.2.1. *Dla zbioru $H(V)$ istnieje zbiór dominujący o rozmiarze $O(s^{-1}n \log n)$ i można go znaleźć w czasie $O(m + ns)$.*

Dowód. Rozpatrujemy dwa przypadki.

Przypadek $H(V) = V$ – wszystkie wierzchołki są wysokostopniowe.

Sprawdzamy problem do pokrycia zbiorów. Dla każdego $v \in V$ definiujemy zbiór $S_v = N(v)$. Rodzina $\mathcal{S} = \{S_v : v \in V\}$ pokrywa całe V .

Pokażemy, że każde pokrycie zbiorów z rodziny $\mathcal{S}$ odpowiada zbiorowi dominującemu tej samej mocy. Niech $\mathcal{C} = \{S_{w_1}, \dots, S_{w_k}\}$ będzie pokryciem V . Definiujemy $D = \{w_1, \dots, w_k\}$. Weźmy dowolny $v \in V$. Skoro $\mathcal{C}$ pokrywa V , istnieje j takie, że $v \in S_{w_j}$. Ponieważ graf jest nieskierowany, relacja sąsiedztwa jest symetryczna: skoro $v \in N(w_j)$, to również $w_j \in N(v)$. A ponieważ $w_j \in D$, mamy $N(v) \cap D \neq \emptyset$ – czyli D jest zbiorem dominującym. Rozumowanie działa też w drugą stronę, więc problemy są równoważne.

Musimy teraz oszacować rozmiar optymalnego pokrycia. Bezpośrednie wyznaczenie optimum jest NP-trudne [19], ale możemy je ograniczyć z góry za pomocą pokrycia ułamkowego – uproszczonego problemu, w którym przypisujemy każdemu zbiorowi S_v dowolną nieujemną wagę x_v . Warunek pokrycia wymaga, aby dla każdego elementu $u \in V$ suma wag zbiorów go zawierających wynosiła co najmniej 1. Przypisujemy każdemu zbiorowi wagę $x_v = 1/s$. Element u należy do zbioru S_v , gdy v jest sąsiadem u lub $v = u$ – takich zbiorów jest dokładnie $|N(u)| = d_u + 1$. Skoro $d_u \geq s$ (bo $H(V) = V$), mamy $|N(u)| \geq s + 1$, więc suma wag zbiorów pokrywających u wynosi $|N(u)| \cdot \frac{1}{s} \geq \frac{s+1}{s} > 1$. Warunek jest spełniony, a rozmiar tego pokrycia ułamkowego wynosi $n \cdot (1/s) = n/s$.

Ponieważ każde pokrycie całkowite jest też pokryciem ułamkowym, zachodzi $\text{OPT} \geq \text{OPT}_{\text{frac}}$. Z naszej konstrukcji $\text{OPT}_{\text{frac}} \leq n/s$, więc $\text{OPT} \leq n/s$. Z twierdzenia Johnsona [20] zachłanny algorytm pokrycia zbiorów daje rozwiązanie o rozmiarze co najwyżej $\text{OPT} \cdot \ln n$, zatem:

$$|D| \leq \frac{n}{s} \cdot \ln n = O(s^{-1} n \log n).$$

Implementacja zachłannego algorytmu z kubełkami posortowanymi według liczby niepokrytych elementów działa w czasie $O(m)$.

Przypadek $H(V) \neq V$ – istnieją wierzchołki niskostopniowe.

Konstruujemy graf $G' = (V', E')$: dodajemy s sztucznych wierzchołków $X = \{x_1, \dots, x_s\}$ tworzących klikę, połączonych z każdym $u \in L(V)$. W G' każdy wierzchołek ma stopień przynajmniej s , więc stosujemy poprzedni przypadek. Żaden wierzchołek z X nie sąsiaduje z $H(V)$, więc $D = D' \cap V$ dominuje $H(V)$ w G . Dodano $O(ns + s^2) = O(ns)$ krawędzi, więc czas wynosi $O(m + ns)$. $\square$

5.3 Opis algorytmu Approx-APSP

Algorytm może też zwracać ścieżki – wystarczy zapamiętywać drzewa BFS z kroków 3 i 5.

5.4 Dowód poprawności

Twierdzenie 5.4.1 ([7], Twierdzenie 4.1). *Dla wszystkich $u, v \in V$ odległości zwracane przez Approx-APSP spełniają:*

$$0 \leq \hat{\delta}(u, v) - \delta(u, v) \leq 2.$$

Dowód. Nierówność $\hat{\delta}(u, v) \geq \delta(u, v)$ zachodzi, ponieważ każda wartość wpisana do $\hat{d}$ odpowiada długości pewnej rzeczywistej ścieżki w G (lub złączenia dwóch ścieżek w ostatnim kroku).

Algorytm 6 Approx-APSP – aproksymacja APSP w grafie nieważonym nieskierowanym

Wejście: Graf $G = (V, E)$, parametr s

Wyjście: Macierz przybliżonych odległości $\hat{D}$

```
1: Zainicjalizuj  $\hat{D}(u, v) \leftarrow \infty$  dla wszystkich  $u, v \in V$ 
2: Oblicz zbiór dominujący  $Dom$  dla  $H(V)$  o rozmiarze  $O(s^{-1}n \log n)$ 
3: for all  $v \in Dom$  do
4:   Wykonaj BFS z  $v$  w pełnym grafie  $G$  i zaktualizuj  $\hat{D}$ 
5: end for
6: for all  $v \in L(V)$  do
7:   Wykonaj BFS z  $v$  w podgrafie  $G[L(V)]$  i zaktualizuj  $\hat{D}$ 
8: end for
9: for all  $u, v \in V \setminus Dom$  do
10:   $\hat{D}[u, v] \leftarrow \min\{\hat{D}[u, v], \min\{\hat{D}[u, w] + \hat{D}[w, v] : w \in D\}\}$ 
11: end for
12: return  $\hat{D}$ 
```

Pokazujemy $\hat{\delta}(u, v) \leq \delta(u, v) + 2$ w trzech przypadkach.

Przypadek 1: $u \in D$. Krok 3 zapewnia dokładne odległości od wierzchołków zbioru dominującego, czyli $\hat{\delta}(u, v) = \delta(u, v)$.

Przypadek 2: $u \in H(V) \setminus D$. Ponieważ D jest zbiorem dominującym dla $H(V)$, istnieje sąsiad $w \in D$ wierzchołka u , a więc $\delta(w, u) = 1$. Odległości od w są dokładne z kroku 3 ($w \in D$), czyli $\hat{\delta}(w, u) = 1$ i $\hat{\delta}(w, v) = \delta(w, v)$. Krok 5 daje:

$$\hat{\delta}(u, v) \leq \hat{\delta}(w, u) + \hat{\delta}(w, v) = 1 + \delta(w, v) \leq 1 + \delta(w, u) + \delta(u, v) = 2 + \delta(u, v),$$

gdzie przedostatnia nierówność wynika z nierówności trójkąta.

Przypadek 3: $u \in L(V)$. Ustalmy najkrótszą ścieżkę P z u do v .

- (a) P przebiega w $L(V)$: krok 4 daje $\hat{\delta}(u, v) = \delta(u, v)$.
- (b) P przechodzi przez $w \in D$: odległości od w są dokładne, więc $\hat{\delta}(u, v) \leq \delta(w, u) + \delta(w, v) = \delta(u, v)$, co implikuje równość.
- (c) P przechodzi przez $w \in H(V) \setminus D$: wierzchołek w ma sąsiada $x \in D$ z $\delta(w, x) = 1$. Odległości od x są dokładne. W ostatnim kroku:

$$\hat{\delta}(u, v) \leq \delta(x, u) + \delta(x, v) \leq (\delta(w, u) + 1) + (\delta(w, v) + 1) = \delta(w, u) + \delta(w, v) + 2 = \delta(u, v) + 2,$$

gdzie ostatnia równość wynika z tego, że w leży na najkrótszej ścieżce P .

□

5.5 Analiza złożoności

Twierdzenie 5.5.1 ([7], Twierdzenie 4.1). *Approx-APSP działa w czasie $O(n^2s + n^3s^{-1} \log n)$.*

Dowód. Analizujemy koszt poszczególnych kroków algorytmu:

- **Krok 1** Inicjalizacja $\hat{\delta}$: $O(n^2)$.

- **Krok 2** Konstrukcja zbioru dominującego D : $O(m + ns)$.
- **Krok 3** BFS z każdego $v \in D$ w grafie G : $|D| = O(s^{-1}n \log n)$ operacji BFS, każda po $O(m)$, łącznie $O(ms^{-1}n \log n)$.
- **Krok 4** BFS z każdego $v \in L(V)$ w podgrafie $G[L(V)]$: co najwyżej n operacji BFS w grafie o $O(ns)$ krawędziach, łącznie $O(n^2s)$.
- **Krok 5** Aktualizacja przez zbiór dominujący: $O(n^2) \cdot |D| = O(n^3s^{-1} \log n)$.

Dominującymi składowymi są $O(n^2s)$ oraz $O(n^3s^{-1} \log n)$, co daje łączną złożoność $O(n^2s + n^3s^{-1} \log n)$. $\square$

Wniosek 5.5.2. Dla $s = \sqrt{n \log n}$ algorytm *Approx-APSP* działa w czasie $O(n^{2.5} \sqrt{\log n})$.

Dowód. Aby zminimalizować $O(n^2s + n^3s^{-1} \log n)$ względem s , przyrównujemy oba składniki:

$$n^2s = n^3s^{-1} \log n \implies s^2 = n \log n \implies s = \sqrt{n \log n}.$$

Podstawiając:

$$n^2 \cdot \sqrt{n \log n} + n^3 \cdot \frac{\log n}{\sqrt{n \log n}} = n^{2.5} \sqrt{\log n} + n^{2.5} \sqrt{\log n} = O(n^{2.5} \sqrt{\log n}).$$

Krok 3 daje $O(m\sqrt{n \log n})$, co dla $m = \Theta(n^2)$ nie dominuje. $\square$

Rozdział 6

Implementacja i porównanie

6.1 Wprowadzenie

Opisane wcześniej algorytmy zostały zaimplementowane w języku C++ zgodnie z przedstawionymi pseudokodami. Do implementacji struktur danych, między innymi grafów, wykorzystano biblioteki NetworKit oraz Koala. Kod źródłowy został dołączony do repozytorium Koala-NetworKit na platformie GitHub.

Wszystkie cztery algorytmy rozwiązują problem najkrótszych ścieżek pomiędzy wszystkimi parami wierzchołków. Trzy z nich (Seidla, USP oraz Spiry) wyznaczają odległości dokładne, natomiast algorytm Aingwortha oblicza aproksymację addytywną: dla każdej pary wierzchołków (u, v) zwrócona wartość $\hat{\delta}(u, v)$ spełnia $\delta(u, v) \leq \hat{\delta}(u, v) \leq \delta(u, v) + 2$. Algorytmy Seidla, USP oraz Aingwortha działają na grafach nieskierowanych i nieważonych, natomiast algorytm Spiry obsługuje grafy ważone o nieujemnych wagach krawędzi.

6.2 Szczegóły implementacji

Algorytmy zostały zaimplementowane w następujących plikach repozytorium Koala:

- `include/shortest_path/Seidel.hpp`, `cpp/shortest_path/Seidel.cpp` – algorytm Seidla (APD), oparty na mnożeniu macierzy sąsiedztwa;
- `include/shortest_path/USP.hpp`, `cpp/shortest_path/USP.cpp` – algorytm USP, oparty na powtarzanym mnożeniu macierzy boolowskich i bitowym odzyskiwaniu odległości;
- `include/shortest_path/Aingworth.hpp`, `cpp/shortest_path/Aingworth.cpp` – aproksymacyjny algorytm Aingwortha, Chekuriego, Indyka i Motwaniego (błąd $+2$);
- `include/shortest_path/Spira.hpp`, `cpp/shortest_path/Spira.cpp` – algorytm Spiry dla grafów ważonych;
- `benchmark/benchmarkAPSP.cpp` – środowisko testowe (benchmark) uruchamiające algorytmy na grafach z generatorów oraz porównujące ich wyniki i czasy działania.

6.3 Wyniki

6.3.1 Zbiory danych

Grafi pochodzą z generatorów dostępnych w bibliotece NetworKit, uzupełnionych o własny generator siatki dwuwymiarowej. Poszczególne rodziny grafów dobrano tak, aby pokrywały różne, istotne z punktu widzenia badanych algorytmów, własności struktury grafu: gęstość, średnicę oraz rozkład stopni. Dokładniej, użyto następujących rodzin:

Graf losowy Erdősa–Rényiego $G(n, p)$. Model grafu losowego, w którym każda z $\binom{n}{2}$ możliwych krawędzi występuje niezależnie z prawdopodobieństwem p . W eksperymentach użyto wariantu *rzadkiego* ($p = 4/n$, dającego średni stopień wierzchołka bliski 4) oraz *gęstego* ($p = 0,5$, w którym istnieje około połowy wszystkich możliwych krawędzi). Grafi te charakteryzują się niewielką średnicą, rzędu $\Theta(\log n)$, i stanowią naturalny punkt odniesienia.

Graf bezskalowy Barabásiego–Alberta. Model grafu o tzw. *rozkładzie potęgowym stopni* (ang. *scale-free*). Graf powstaje przez stopniowe dodawanie wierzchołków, z których każdy łączy się z już istniejącymi wierzchołkami z prawdopodobieństwem proporcjonalnym do ich stopnia (mechanizm *preferencyjnego dołączania*). W efekcie powstaje niewiele wierzchołków o bardzo wysokim stopniu (tzw. *koncentratory*, ang. *hubs*) oraz wiele wierzchołków o niskim stopniu. Model ten dobrze odwzorowuje strukturę wielu sieci rzeczywistych, takich jak sieć WWW czy sieci społecznościowe.

Graf „małego świata” Watts–Strogatza. Model *małego świata* (ang. *small-world*). Powstaje z regularnego pierścienia przez losowe „przepięcie” części krawędzi. Grafi takie łączą dwie cechy: wysoki *współczynnik klasteryzacji* (sąsiedzi danego wierzchołka są zwykle również swoimi sąsiadami, co odpowiada lokalnej strukturze) oraz małą średnią odległość między wierzchołkami (dowolne dwa wierzchołki dzieli zwykle niewiele krawędzi). Model odwzorowuje wiele sieci rzeczywistych, w których mimo lokalnej organizacji odległości globalne pozostają krótkie.

Pierścień regularny (ang. *ring lattice*). Wierzchołki ułożone są na okręgu, a każdy z nich połączony jest z ustaloną liczbą najbliższych sąsiadów po obu stronach. Jest to graf regularny (wszystkie wierzchołki mają ten sam stopień) o bardzo dużej średnicy, rzędu $\Theta(n)$. Stanowi przypadek skrajny, obciążający algorytmy zależne od średnicy grafu.

Siatka dwuwymiarowa (ang. *grid*). Regularna siatka prostokątna, w której wierzchołki tworzą kratę, a krawędzie łączą sąsiadów w pionie i poziomie. Graf ten jest planarny, ma ograniczony stopień (co najwyżej 4) oraz średnicę rzędu $\Theta(\sqrt{n})$. Jego dodatkową zaletą jest to, że odległości między wierzchołkami dane są jawnym wzorem (metryka miejska, tzw. *odległość Manhattan*), co pozwala niezależnie zweryfikować poprawność algorytmów. Generator siatki został zaimplementowany samodzielnie, gdyż biblioteka NetworKit nie udostępnia go bezpośrednio.

Graf klastrowy (ang. *clustered*). Graf z wyraźną strukturą społecznościową: wierzchołki podzielone są na grupy (klastry), przy czym prawdopodobieństwo istnienia krawędzi wewnątrz grupy jest znacznie wyższe niż między grupami. Model odwzorowuje sieci o naturalnym podziale na wspólnoty.

Sieci drogowe stanów USA (*roads*). Zestaw obejmuje grafy planarne reprezentujące sieci dróg głównych wybranych stanów Stanów Zjednoczonych, pozyskane z repozytorium All-pairs-shortest-paths (oryginalne dane DIMACS). Każdy wierzchołek odpowiada węzłowi drogowemu, a każda krawędź – odcinkowi drogi o wadze równej jej długości. Grafy są planarne. Rozmiary wahają się od $n = 38$ (Hawaje, HI) do $n = 2002$ (Pensylwania, PA). Algorytm Spiry użył oryginalnych wag krawędzi; pozostałe algorytmy pracowały na grafach nieważonych (wagi zignorowano).

Dla każdej rodziny generowano grafy o rosnącej liczbie wierzchołków n . Ponieważ algorytmy Seidla oraz USP operują na gęstych macierzach o rozmiarze $n \times n$ (algorytm USP przechowuje jednocześnie kilkadziesiąt takich macierzy), ich zapotrzebowanie na pamięć rośnie kwadratowo z n ; z tego powodu wszystkie klasy grafów były przetestowane na grafach z maksymalnie 5000 wierzchołków.

Ze względu na to, że badane algorytmy zakładają spójność grafu wejściowego, z każdego wygenerowanego grafu wyodrębniano największą spójną składową, a jej wierzchołki przenumerowywano na kolejne liczby całkowite. Dla algorytmu Spiry, wymagającego grafu ważonego, krawędziom przypisywano wagi losowane z rozkładu jednostajnego na przedziale $[1, 100]$. Poprawność każdej implementacji weryfikowano przez porównanie wyznaczonych odległości z wynikami niezależnej implementacji przeszukiwania wszerz (BFS, dla grafów nieważonych) oraz algorytmu Dijkstry (dla grafów ważonych); w przypadku algorytmu Aingwortha sprawdzano dodatkowo, czy zwrócone wartości mieszczą się w gwarantowanym przedziale błędu addytywnego $+2$.

Wszystkie eksperymenty przeprowadzono na komputerze osobistym wyposażonym w procesor Apple M1 (8 rdzeni) oraz 8 GB pamięci RAM, na systemie macOS 26.5. Kod skompilowano kompilatorem Apple Clang 17 w standardzie C++20 z optymalizacją `-O2`.

6.3.2 Empiryczna złożoność czasowa

Dopasowując zależność $t \sim n^\alpha$ otrzymano wykładniki zebrane w tabeli 6.1. Pokrywają się one z przewidywaniami teoretycznymi. Algorytmy macierzowe rosną jak n^3 (biblioteka Eigen nie wykorzystuje szybkiego mnożenia macierzy, dlatego nie obserwuje się wykładnika mniejszego niż 3). Algorytmy Aingwortha i Spiry rosną istotnie wolniej – na grafach rzadkich, dla których zbiór wierzchołków wysokiego stopnia jest pusty, algorytm Aingwortha degeneruje się do n przeszukiwań wszerz, stąd wykładnik bliski 2.

Tabela 6.1: Empiryczny wykładnik α w zależności $t \sim n^\alpha$, wyznaczony po odcięciu najmniejszych rozmiarów (poniżej 500 wierzchołków), oraz złożoność teoretyczna.

Algorytm	α (empiryczny)	złożoność teoretyczna
Seidel	2,9–3,2	$O(n^3)$
USP	3,1–3,2	$O(n^3 \log n)$
Aingworth	2,1–2,4	$O(n^{2,5} \sqrt{\log n})$
Spira	2,0–2,3	$O(n^2 \log n)$

6.3.3 Porównanie czasów działania

Rysunek 6.1 przedstawia czasy działania dla czterech rodzin grafów w funkcji liczby wierzchołków n (skala logarytmiczna na obu osiach). Najważniejsze obserwacje, dla $n \approx 5000$, są następujące:

- Algorytm **USP** jest bezwzględnie najwolniejszy, a jego czas jest niemal *niezależny od rodziny grafu* (około 295 s dla każdej klasy) – koszt zależy wyłącznie od n , a nie od liczby krawędzi. Jest on 5–70 razy wolniejszy od algorytmu Seidla i 425–1034 razy wolniejszy od algorytmu Aingwortha. Mimo że asymptotycznie zbliżony do algorytmu Seidla, ma wielokrotnie większą stałą (odzyskiwanie kolejnych bitów odległości wymaga kilkudziesięciu mnożeń macierzy zamiast kilku).
- Algorytm **Aingwortha** jest najszybszy w każdej klasie (0,27–0,70 s przy $n \approx 5000$). Na grafach gęstych bije nawet naiwne wyznaczanie APSP za pomocą n przeszukiwań wszerz: 472 ms wobec 53 s, czyli około 100-krotnie szybciej.
- Algorytm **Seidla** wyprzedza pozostałe algorytmy dokładne *tylko na grafach gęstych* (4,2 s wobec 22,8 s dla algorytmu Spiry), ponieważ jego koszt nie zależy od liczby krawędzi. Na grafach rzadkich zależność się odwraca: przy $n = 5000$ algorytm Seidla (19,9 s) jest wolniejszy od algorytmu Spiry (12,2 s), a na siatce dwuwymiarowej, o dużej średnicy, różnica jest znaczna (55,4 s wobec 5,7 s). Wynika to z dwóch efektów: algorytm Spiry korzysta z rzadkości grafu, natomiast algorytm Seidla wykonuje tym więcej poziomów rekursji, im większa jest średnica grafu.

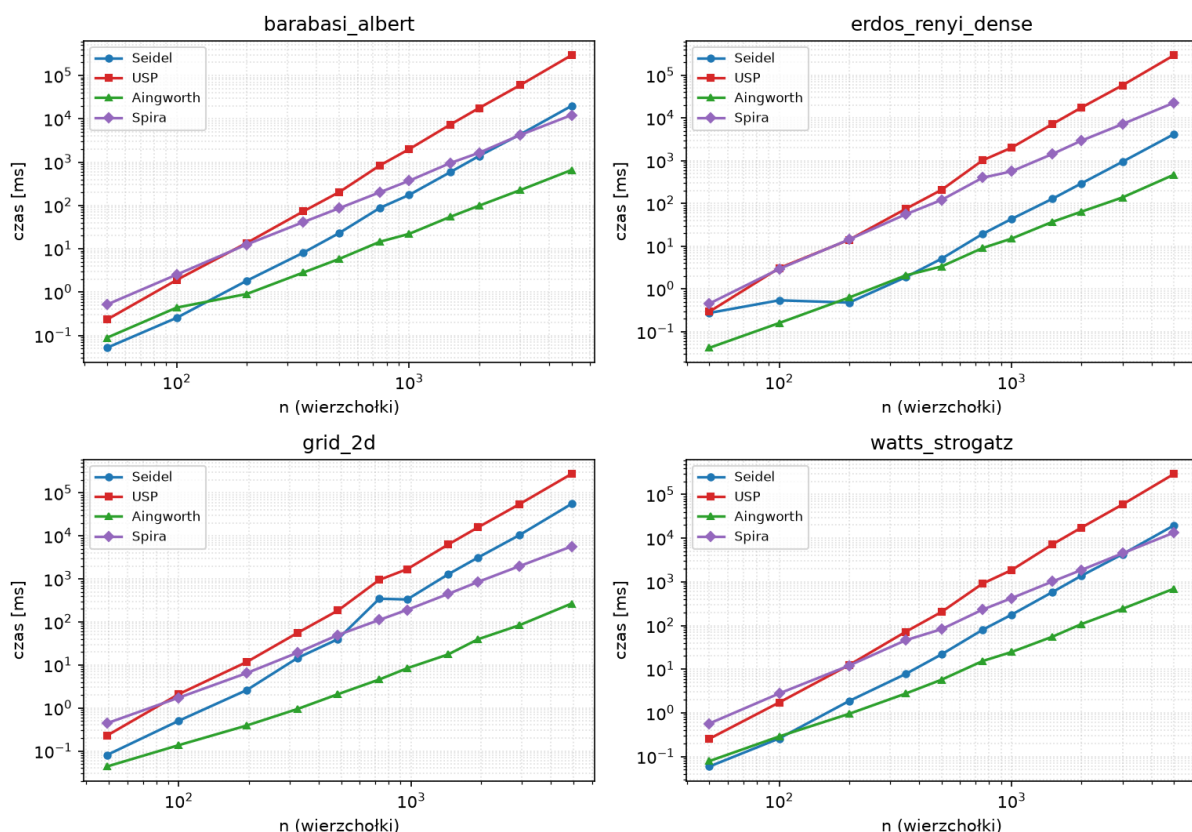
Graf	n	m	Seidel [ms]	USP [ms]	Aingworth [ms]	Spira [ms]
HI	38	43	0.07	0.18	0.04	0.10
DE	148	216	1.61	5.27	0.29	1.74
NH	197	275	3.18	13.01	0.59	4.42
ME	221	318	5.03	16.92	0.73	5.68
AZ	225	308	4.85	19.11	0.76	6.13
MA	381	571	23.32	98.78	2.04	20.08
MN	798	1222	188.7	942.3	9.11	102.7
VA	886	1340	292.7	1225.5	10.26	119.7
CA	935	1373	354.3	1499.0	11.81	125.8
NC	948	1432	390.9	1579.2	13.51	139.3
OH	998	1620	367.6	1763.4	15.35	176.7
IL	1299	1997	811.4	4406.2	25.55	292.8
FL	1317	1894	954.3	4523.4	20.93	246.4
NY	1437	2271	1283.1	5939.0	26.06	337.0
GA	1557	2508	1575.6	7689.6	37.72	439.8
TX	1755	2758	2260.7	10853	46.12	628.1
PA	2002	2898	3243.7	15904	64.92	638.0

Tabela 6.2: Czasy działania algorytmów APSP na grafach z zestawu roads.

Tabela 6.2 prezentuje wyniki dla sieci drogowych 17 stanów USA, posortowanych rosnąco według liczby wierzchołków n .

Każdy z przebiegów algorytmu Aingwortha wykazał zerowy błąd. Wynika to ze struktury grafów. Algorytm Aingwortha wyznacza zbiór dominujący oparty na wierzchołkach

Czas działania APSP w zależności od n (per rodzina grafu)



Rysunek 6.1: Czas działania algorytmów APSP w zależności od liczby wierzchołków n dla czterech rodzin grafów (skala logarytmiczna na obu osiach). Wszystkie cztery algorytmy uruchomiono dla rozmiarów do $n = 5000$.

wysokiego stopnia. Badane grafy były rzadkie, a stopień każdego wierzchołka był ograniczony, więc zbiór dominujący jest pusty i algorytm degeneruje się do n przeszukiwań wszerek – zachowując się identycznie jak naiwna metoda BFS.

Ranking czasów jest spójny z wynikami dla pozostałych grafów rzadkich: najszybszy jest Aingworth (0,04–65 ms), następnie Spira (0,10–638 ms), dalej Seidel (0,07–3244 ms), a algorytm USP jest zdecydowanie najwolniejszy (0,18–15904 ms).

Wzrost czasów działania wraz z n jest zgodny z przewidywaniami teoretycznymi. Algorytm Seidla – mimo teoretycznej złożoności $O(n^\omega \log n)$ – w praktyce wypada gorzej od Spiry dla grafów rzadkich, gdyż opiera się na mnożeniu gęstych macierzy, co jest nieefektywne gdy $m \ll n^2$.

Ranking algorytmów zależy zatem od struktury grafu: na grafach gęstych o małej średnicy przewagę ma podejście macierzowe (Seidel), a na grafach rzadkich – podejście oparte na pojedynczych źródłach (Spira). Algorytm Aingwortha, kosztem dopuszczenia błędu addytywnego $+2$, jest najszybszy niezależnie od klasy grafu.

6.3.4 Dokładność aproksymacji Aingwortha

Choć algorytm gwarantuje błąd co najwyżej $+2$, w praktyce błąd ten pojawiał się jedynie dla części grafów:

- graf gęsty Erdősa–Rényiego – błąd +2 wystąpił dla każdego badanego rozmiaru;
- graf bezskalowy Barabásiego–Alberta – błąd +2 jedynie dla najmniejszego grafu ($n = 50$);
- graf małego świata oraz siatka dwuwymiarowa – wynik był zawsze dokładny.

Błąd addytywny koreluje z gęstością i małą średnicą grafu: im krótsze odległości i im więcej par wierzchołków znajduje się w tej samej odległości, tym częściej zaokrąglenie wewnątrz algorytmu prowadzi do przeszacowania. Na grafach rzadkich o dużej średnicy algorytm Aingwortha zachowywał się w przeprowadzonych testach jak metoda dokładna.

6.3.5 Uwagi dotyczące zużycia pamięci

Algorytmy Seidla i USP operują na gęstych macierzach $n \times n$. Algorytm USP przechowuje ich kilkadziesiąt jednocześnie, co teoretycznie odpowiada około 8 GB pamięci dla $n = 5000$. W praktyce maksymalne zużycie pamięci całego programu wyniosło około 1,3 GB dla wszystkich instancji testowych oprócz grafów internetowych *AS* z repozytorium All-pairs-shortest-paths, które zawierają około 15 000 wierzchołków. Dla tych grafów zużycie pamięci przekroczyło 18 GB, co uniemożliwiło przeprowadzenie testów na dostępnym sprzęcie.

6.3.6 Wnioski

Zebrane wyniki prowadzą do następujących wniosków praktycznych:

1. Algorytm **USP** ma przede wszystkim znaczenie teoretyczne – jego stała jest na tyle duża, że pozostaje niepraktyczny nawet dla umiarkowanych wartości n .
2. Algorytm **Seidla** opłaca się na grafach gęstych o małej średnicy; na grafach rzadkich lub o dużej średnicy ustępuje prostszemu algorytmowi **Spiry**.
3. Algorytm **Aingwortha** był najszybszy we wszystkich testowanych klasach grafów i często dokładny; stanowi atrakcyjny wybór, gdy dopuszczalny jest niewielki błąd addytywny.

Bibliografia

- [1] Edsger Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [2] Robert Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962.
- [3] Philip Spira. A new algorithm for finding all shortest paths in a graph of positive arcs in average time $O(n^2 \log^2 n)$. *SIAM Journal on Computing*, 2(1):28–32, 1973.
- [4] Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. *Journal of Computer and System Sciences*, 51(3):400–403, 1995.
- [5] Avi Shoshan and Uri Zwick. All pairs shortest paths in undirected graphs with integer weights. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS 1999)*, pages 605–614. IEEE Computer Society, 1999.
- [6] Pavlos Eirinakis, Matthew Williamson, and Krishnamurthy Subramani. On the Shoshan–Zwick algorithm for the all-pairs shortest path problem. *Journal of Graph Algorithms and Applications*, 21(2):177–181, 2017.
- [7] Donald Aingworth, Chandra Chekuri, Piotr Indyk, and Rajeev Motwani. Fast estimation of diameter and shortest paths (without matrix multiplication). *SIAM Journal on Computing*, 28(4):1167–1181, 1999.
- [8] Udaya Kumar Reddy. A survey of the all-pairs shortest paths problem and its variants in graphs. *Acta Universitatis Sapientiae, Informatica*, 8(1):16–40, 2016.
- [9] Ulrik Brandes. A faster algorithm for betweenness centrality. *The Journal of Mathematical Sociology*, 25(2):163–177, 2001.
- [10] Sven Peyer, Dieter Rautenbach, and Jens Vygen. A generalization of Dijkstra’s shortest path algorithm with applications to VLSI routing. *Journal of Discrete Algorithms*, 7(4):377–390, 2009.
- [11] Michael Fredman and Robert Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science (FOCS 1984)*, pages 338–346. IEEE Computer Society, 1984.
- [12] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

- [13] Donald Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977.
- [14] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. More asymmetry yields faster matrix multiplication. In Yossi Azar and Debmalya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2025)*, pages 2005–2039. Society for Industrial and Applied Mathematics, 2025.
- [15] Kenneth Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill Education, New York, NY, 8 edition, 2019.
- [16] Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein. *Wprowadzenie do algorytmów*. Wydawnictwo Naukowe PWN, Warszawa, 7 edition, 2012.
- [17] Noga Alon, Zvi Galil, and Oded Margalit. On the exponent of the all pairs shortest path problem. *Journal of Computer and System Sciences*, 54(2):255–262, 1997.
- [18] Leslie Kerr. *The effect of algebraic structure on the computational complexity of matrix multiplication*. PhD thesis, Cornell University, Ithaca, NY, 1970.
- [19] Richard Karp. Reducibility among combinatorial problems. In Raymond Miller and James Thatcher, editors, *Complexity of Computer Computations*, The IBM Research Symposia Series, pages 85–103, New York, 1972. Plenum Press.
- [20] David Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.